AER 1515 – Perception for Robotics

Lecture 3 – Machine Learning

Prof. Steven Waslander



Institute for Aerospace Studies
UNIVERSITY OF TORONTO

Machine Learning Improvements

- ML
  - Fix maximum likelihood to be general, then connect to least squares
  - Add maximum a posteriori, by including a prior on the parameter distributions with limited data
  - Expand list of other ideas from classic ML to point people to, and the reasons they are interesting
  - Connect these examples to the course.

- NNs
  - Improve explanation and positioning of validation set and train/val split (move to ML)
  - Cut down backprop, gradient descent to two slides
  - Update slides to highlight the pieces and workflow in CNN tutorial
- Convs
  - Actually do conv tutorials and add one for robotic vision in pytorch
  - Update slides to highlight the pieces and workflow in tutorial
- RNNs
  - Slightly expand backprop through time
  - Add examples from robotics, language, tracking
- Transformers
  - Add a slide or two describing the idea
  - Show results on vision tasks/lidar
- Diffusion
  - Add a slide or two describing the idea
  - Show some results on vision/lidar tasks

Outline

- **Machine Learning**
- Neural networks
- Convolutional Neural Networks
- Recurrent Neural Networks
- Transformers

Machine Learning

> "A computer program is said to learn from experience, *E*, with respect to some class of tasks, *T*, and performance measure, *P*, if its performance at tasks in *T*, as measured by *P*, improves with experience, *E*."
>
> [Machine Learning - Mitchell 1997]

- Tasks, *T*
  - Classification, Regression, Segmentation, Object Detection, Transcription, Translation, Anomaly Detection, Clustering, Imputation, Denoising, Captioning

- Experience, *E*
  - Datasets with many data points, usually fixed in advance
  - Labelled or unlabeled, leading to supervised or unsupervised learning
  - Non-fixed datasets occur in reinforcement learning where algorithm interacts with environment

- Performance measure, *P*
  - Accuracy, error rate, average precision, uncertainty error
  - Training and test set performance evaluation

Linear Regression

- Linear regression (task)
  - Given sample input points $x_i \sim x$ and outcomes $y_i \sim y$ (experiences), we seek to find a linear relationship between random variables x and y which must hold approximately for each sample

$$y_i = b + A_1 x_{i1} + A_2 x_{i2} + \ldots + \epsilon_i, \; \forall i \in \{1, \ldots, n\}$$

  - The ordinary least squares version of linear regression seeks to find parameters $\theta = (b \; A_1 \; A_2 \; \ldots)^{\mathrm{T}}$ that minimize the mean square error (MSE) (performance measure) between measured and expected values of y

$$\theta^* = \arg\min_{\theta} \frac{1}{N} \sum_{i=1}^{N} ||y_i - b - A_1 x_{i1} + \ldots||^2$$

- Let $X = \begin{bmatrix} 1 & x_{11} & \ldots \\ 1 & x_{21} & \ldots \\ & \ldots & \end{bmatrix}$ and $Y = \begin{bmatrix} y_1 \\ y_2 \\ \ldots \end{bmatrix}$

- The well-known solution to the linear regression problem is: $\theta^* = (X^T X)^{-1} X^T Y$

Scikit-Learn Diabetes Dataset Example

- Diabetes dataset – 442 records, 10 numerical features, no missing data
  - Use single feature regression to predict quantitative progression of disease
    - Feature 2 = Body mass index

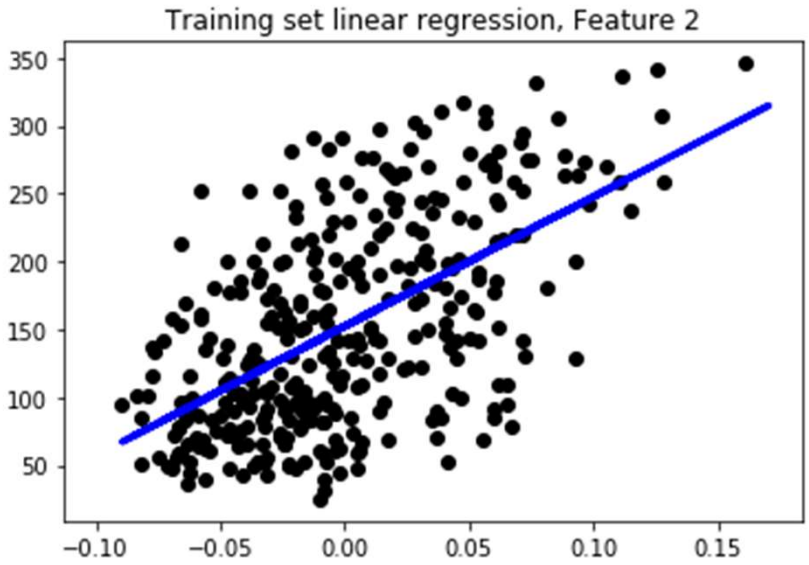
```
# Load the diabetes dataset
diabetes = datasets.load_diabetes()

# Use only one feature
diabetes_X = diabetes.data[:, np.newaxis, 2]

# Create linear regression object
regr = linear_model.LinearRegression()

# Train the model using the training sets
regr.fit(diabetes_X, diabetes_y)

# Make predictions using the testing set
diabetes_y_pred = regr.predict(diabetes_X)
```



Maximum Likelihood Estimation for Linear Regression

- Assume that the error term in our linear regression is drawn from a Gaussian distribution, and that y is again a linear combination of x, we can write

$$\epsilon_i = y_i - b - Ax_i \sim \mathcal{N}(0, \Sigma_\epsilon)$$

  - We can express the probability distribution for y given x and parameters $\theta = \{b, A\}$ as

$$p(y|x, \theta)$$

  - To identify strong parameters, we want to identify the maximum likelihood estimate for y, that is, we want to identify parameters, θ, that maximize the likelihood of the outcomes y given inputs x,

$$\hat{\theta}_{ML} = \arg\max_\theta p(y|x, \theta)$$

  - Given that we have n independent and identically distributed samples, we get

$$\hat{\theta}_{ML} = \arg\max_\theta \prod_{i=1}^{n} p(y_i|x_i, \theta)$$

Maximum Likelihood Estimation

- As usual, we take the log and switch to minimization - the negative log likelihood (NLL)

$$\hat{\theta}_{ML} = \arg\max_{\theta} \prod_{i=1}^{n} p(y_i|x_i,\theta) = \arg\min_{\theta} -\sum_{i=1}^{n} \log p(y_i|x_i,\theta)$$

- If x_i is known, the conditional distribution for y_i is defined by the error between measured and predicted values for y_i, which we can assume is Gaussian (isn't everything?)

$$\begin{aligned} p(y_i|x_i,\theta) &= \mathcal{N}(y_i - Ax_i - b, \Sigma_\epsilon) \\ &= \frac{1}{\sqrt{\det(2\pi\Sigma_\epsilon)}} e^{-\frac{1}{2}(y_i - Ax_i - b)^T (\Sigma_\epsilon)^{-1} (y_i - Ax_i - b)} \end{aligned}$$

Maximum Likelihood Estimation

- Substituting in, we see the negative log likelihood (NLL) becomes

$$\hat{\theta}_{ML} = \arg\min_{\theta} -\sum_{i=1}^{n} \log \frac{1}{\sqrt{\det(2\pi\Sigma_\epsilon)}} e^{-\frac{1}{2}(y_i - Ax_i - b)^T(\Sigma_\epsilon)^{-1}(y_i - Ax_i - b)}$$

- Which simplifies to

$$\hat{\theta}_{ML} = \arg\min_{\theta} \quad \frac{N}{2}\log\sqrt{\det(2\pi\Sigma_\epsilon)} + \sum_{i=1}^{n} \frac{1}{2}(y_i - Ax_i - b)^T(\Sigma_\epsilon)^{-1}(y_i - Ax_i - b)$$

- If we assume identity for the covariance of Σ_ε and we eliminate constants that don't affect the optimization result, we arrive at least squares once again

$$\hat{\theta}_{ML} = \arg\min_{\theta} \sum_{i=1}^{n} ||(y_i - Ax_i - b)||^2$$

Maximum A Posteriori Estimation

- Maximum likelihood but with a prior and Bayes rule is known as maximum a posteriori
  - For linear regression problem studied so far
    - Add prior distribution on parameters θ, can also be Gaussian

$$\theta \sim \mathcal{N}(\mu_\theta, \Sigma_\theta)$$

    - The MAP estimate and model distribution can then be derived for distributions with conjugate priors

$$\theta^* = \arg\max_\theta \eta p(y|x,\theta)p(\theta)$$

  - Can be used to update a model when new samples become available
  - Used to update state estimates with known models for the evolution of the state and measurement probabilities

Learning Process

- Learning seeks to *generalize* its performance over a set of tasks, and not just fit to existing data
  - We are interested in performance on the underlying unknown data distribution:

$$p_{data}(y|x)$$

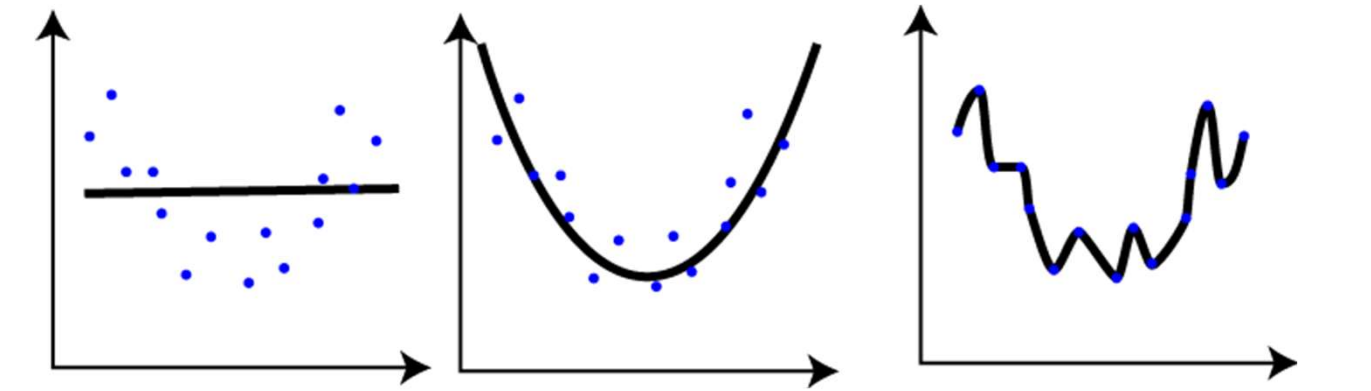
  - We only have access to a finite number of samples from this distribution:

$$(X, y) = \{(x_1, y_1), \ldots, (x_n, y_n)\}$$

  - So we divide data into training and test sets
    - Develop a model from training data through optimization
    - Evaluate on test to determine generalization performance

Bias-Variance Trade-off

- When we fit data, we want our model to have an appropriate capacity.
  - The more 'capacity' our model has (i.e., the more variance it is able to sustain), the more likely it is to overfit to the training data and therefore have poor generalization. However, a model with a capacity too low will not be very useful.



Underfit
(low capacity, high bias)

Just right

Overfit
(high capacity, high variance)

Classical Machine Learning

- Logistic regression
  - Regression to perform a binary classification instead of a continuous estimation
  - Assume y is from a Bernoulli distribution, perform maximum likelihood estimation

$$p(y|x,\theta) = \text{Ber}(y|\sigma(w^T x)) \quad \sigma(z) = \frac{1}{1+\exp^{-z}}$$

  - Becomes classification if output is thresholded to estimate output class

- Decision Trees and Random Forest
  - For multi-variate inputs, create decisions trees with learned thresholds on each variable to branch on
  - Repeat on different sections of the data to create forests and combine outputs

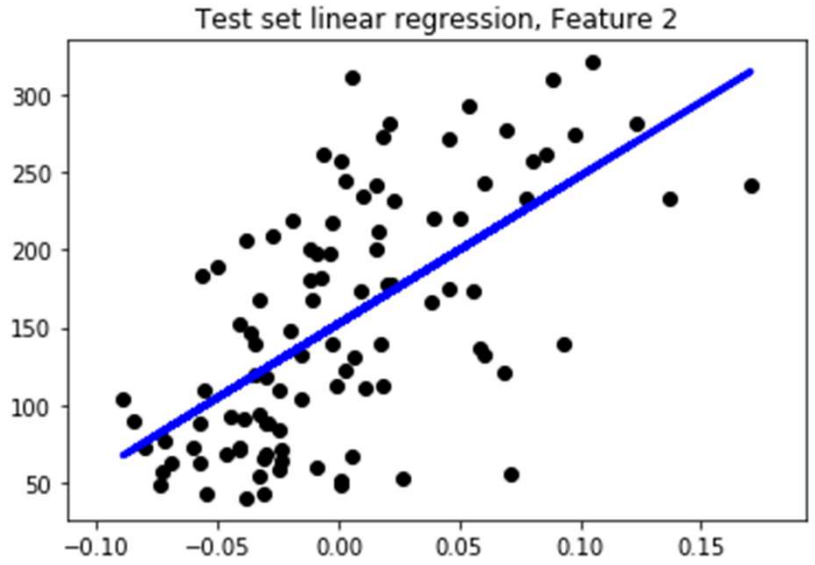
- Boosting
  - Train models one at a time to correct errors of previous models. Combines many weak learning models into a collection that performs better

Scikit-Learn Diabetes Dataset Example

- But in machine learning, we want to know how well we learned the generic task, *T*
  - Predicting disease progression for new patients, given body mass index measurement

  - Divide data into 342 training and 100 test examples

	MSE	Variance
Train	3930.79	0.33
Test	3753.77	0.38

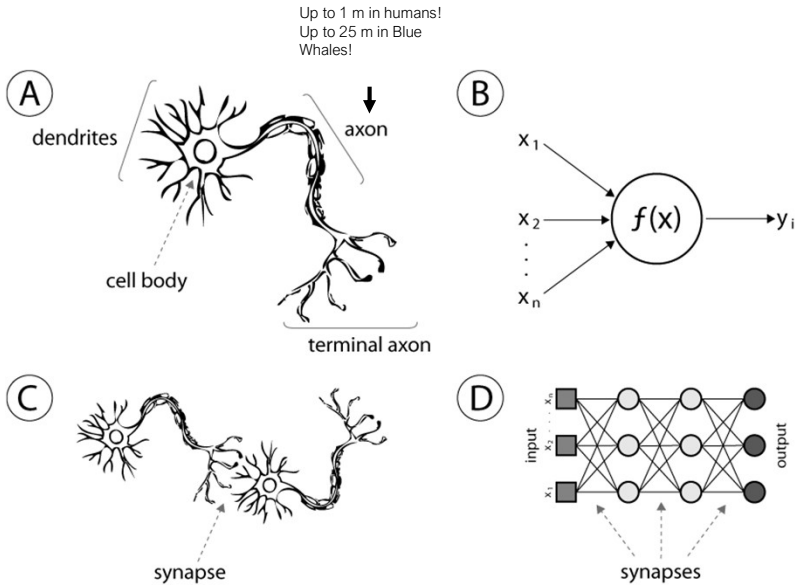


Outline

- Machine Learning
- **Neural networks**
- Convolutional Neural Networks
- Recurrent Neural Networks
- Transformers

What is a Neural Network?

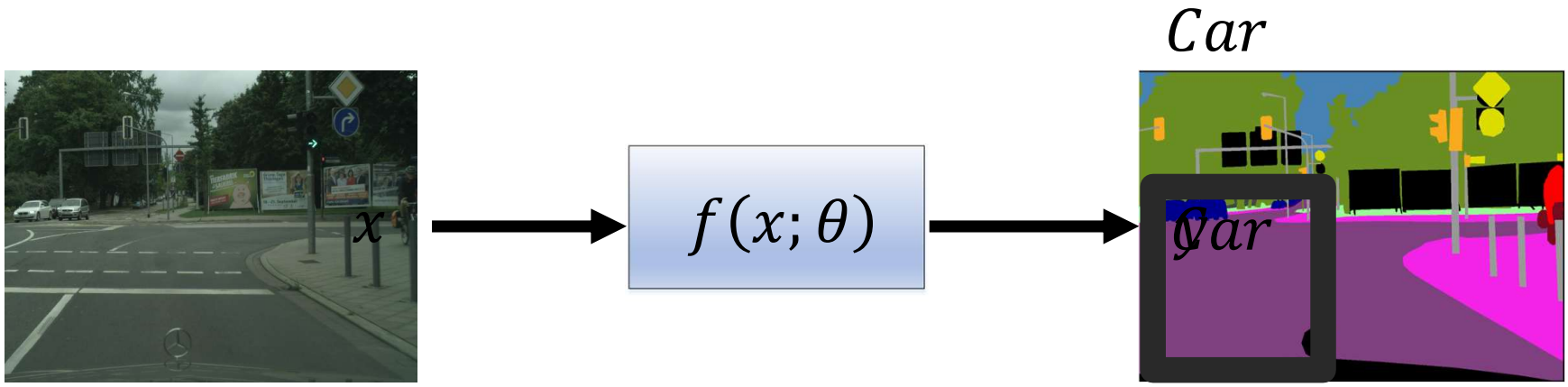
- An ***artificial neural network*** (ANN) is an information processing algorithm ***inspired*** by a simple model of the brain
- G. Hinton: "*the brain must work somehow*"
- It is a network of interconnected but *simple processing elements*, neurons, that are used together to solve a problem



Connections between neurons are weighted, similar to the way in which synapses in the brain have varying strengths

Feedforward Neural Networks

- Functions to estimate:
  - Object Classification: Image → Label
  - Object Detection: Image → Labels + Bounding Boxes
  - Depth Estimation: Image → Depth for every pixel
  - Semantic Segmentation: Image → Label for every pixel



Feedforward Neural Networks

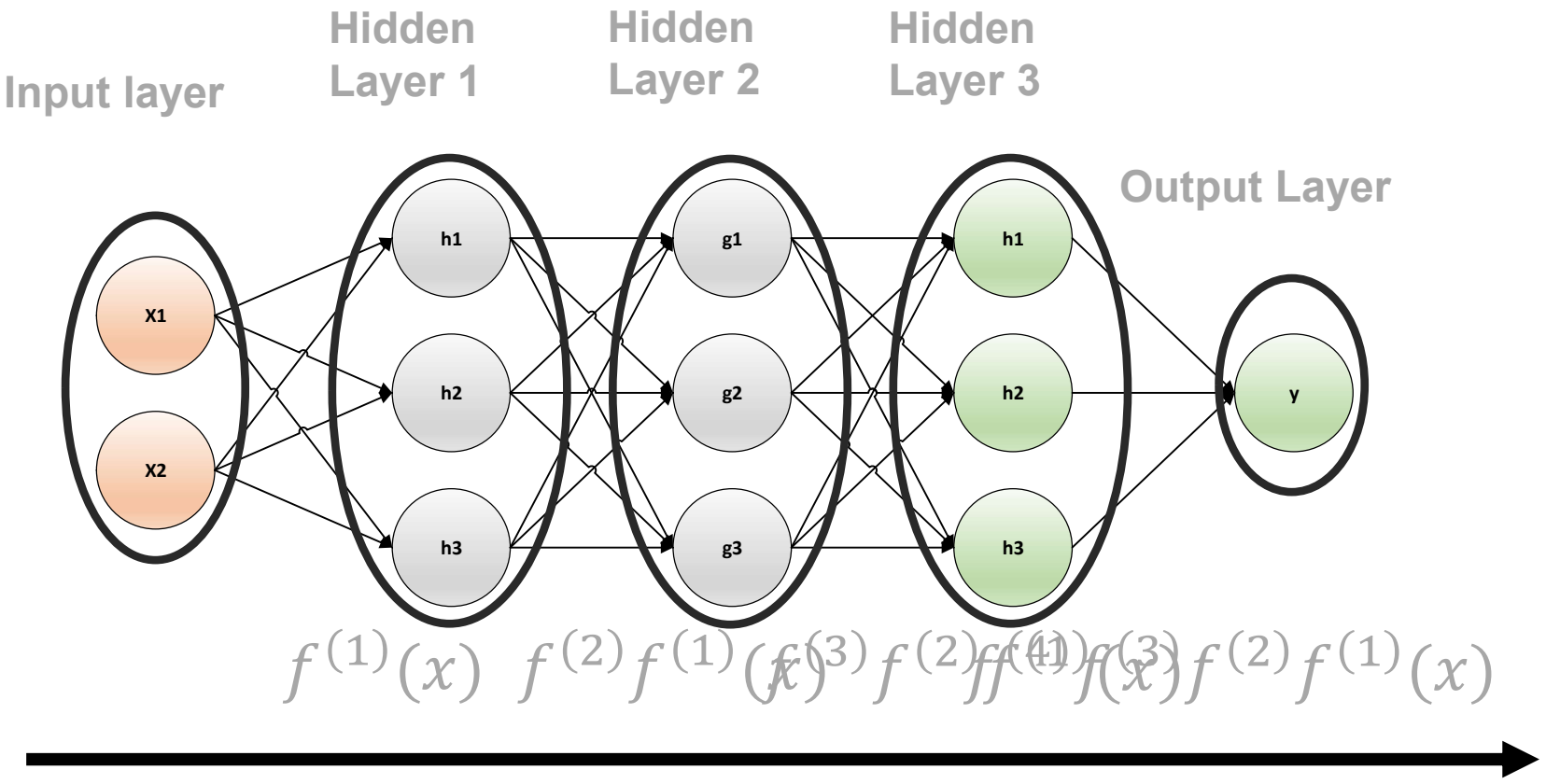
- As with all machine learning models in this course, a *Feedforward Neural Network* (FNN) defines a mapping from input $\boldsymbol{x}$ to output $\boldsymbol{y}$ as:

$$y = f(x; \theta)$$

- An N-layer FNN is represented as the function composition:

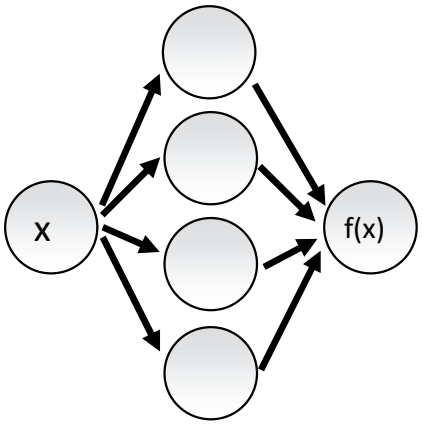
$$f(x; \theta) = f^{(N)}\left(f^{(N-1)}\left(\ldots f^{(2)}\left(f^{(1)}(x)\right)\right)\right)$$

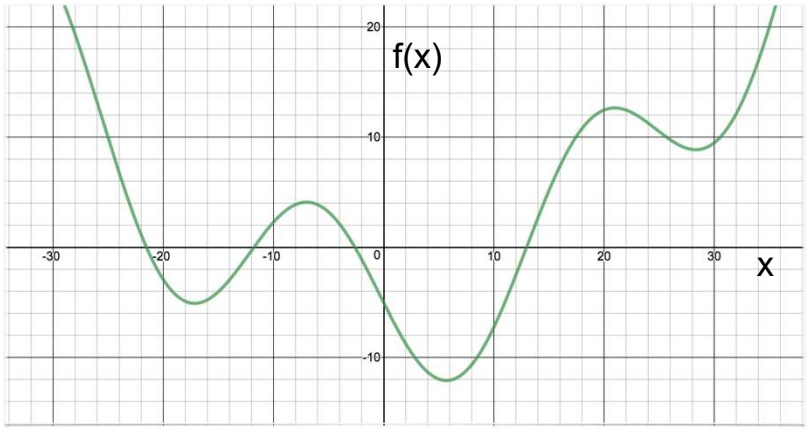
Feedforward Neural Networks



Universal Approximation Theorem

- Neural networks with nonlinear activation functions are universal approximators.
  - A NN with a *single* hidden layer (including a non-linearity) can approximate a given function and its derivatives to arbitrary accuracy (with enough neurons).
  - Intuitively, each neuron can learn a step function. Given enough step functions, we can approximate any function.





Arbitrary function f(x)

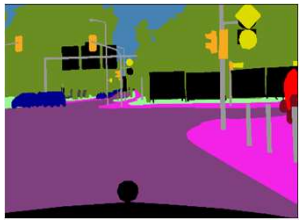
Approximation Capabilities of Muitilayer Feedforward Networks, Hornik et. al. (1991)
http://neuralnetworksanddeeplearning.com/chap4.html

Mode Of Action Of Neural Networks

- Training: Give neural network examples of $f^*(\boldsymbol{x})$. for a wide variation of the input $\boldsymbol{x}$. Then, optimize its parameters $\boldsymbol{\theta}$ to force $f(\boldsymbol{x};\boldsymbol{\theta}) \cong f^*(\boldsymbol{x})$
- Pairs of $\boldsymbol{x}$ and $f^*(\boldsymbol{x})$ are called training data



$\boldsymbol{x}$



$f^*(\boldsymbol{x})$

- Only output is specified by training data! Network is free to do anything with its hidden layers

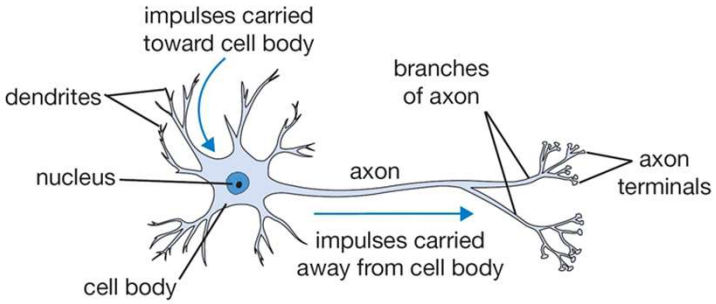
Hidden Units

- Hidden units are a linear combination of inputs passed through an activation function

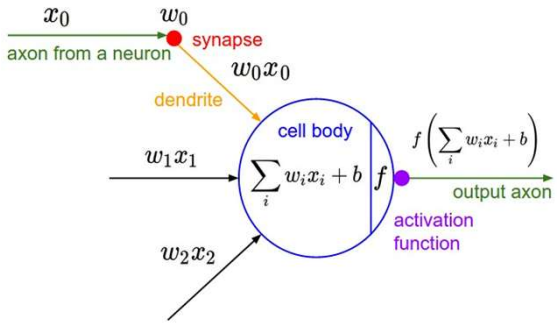
$$h_n = g(W^T h_{n-1} + b)$$

- Activation function g
- Input h_{n-1}
- Weight matrix $\boldsymbol{W}$
- Bias $\boldsymbol{b}$
- Parameters $\boldsymbol{\theta}$ are the weights and biases of all the layers of the network
- Transformed parameters passed through activation function g

Biological Neuron



Artificial Neuron



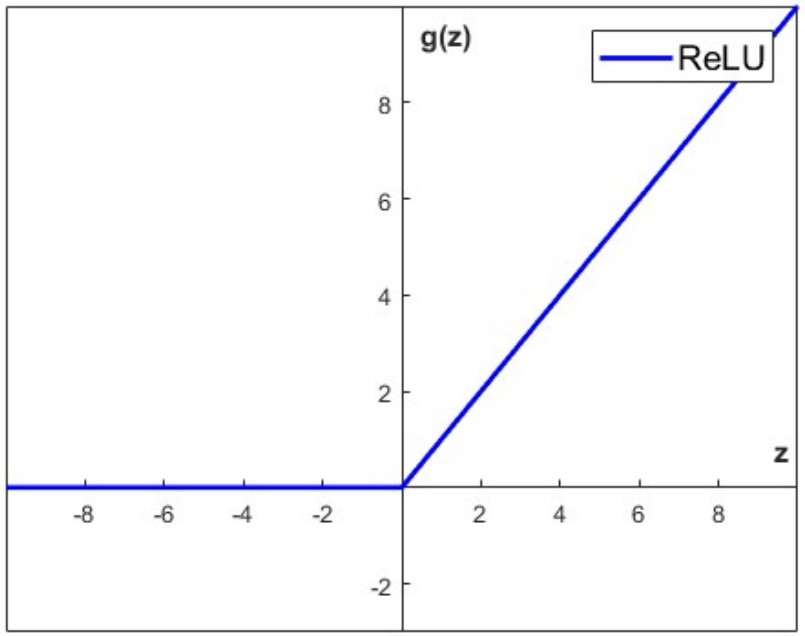
http://cs231n.github.io/neural-networks-1/

The Rectified Linear Unit: ReLU

- The ReLU hidden unit is currently the default choice of activation function for Feedforward Neural Networks

$$g(z) = \max(0, z)$$

- Why? Two distinct regions, one with a stable gradient, the other with no effect.



Example: Hidden Layer With ReLU Activation Function

- $h_{n-1} = \underbrace{\begin{bmatrix} -17 & 12 & -6 \\ -14 & 2 & 7 \end{bmatrix}}_{\mathbf{2 \times 3}}$, $W = \underbrace{\begin{bmatrix} 9 & -2 & 9 & 10 & -3 \\ -3 & 10 & 4 & 6 & 3 \end{bmatrix}}_{\mathbf{2 \times 5}}$, $b = \begin{bmatrix} -2 & -18 & -12 \\ -6 & -3 & 2 \\ -7 & 4 & -9 \\ -6 & -12 & -10 \\ -8 & 4 & -15 \end{bmatrix}$

- Evaluate affine transformation $W^T h_{n-1} + b$:

$$\begin{bmatrix} 9 & -3 \\ -2 & 10 \\ 9 & 4 \\ 10 & 6 \\ -3 & 3 \end{bmatrix} \begin{bmatrix} -17 & 12 & -6 \\ -14 & 2 & 7 \end{bmatrix} + \begin{bmatrix} -2 & -18 & -12 \\ -6 & -3 & 2 \\ -7 & 4 & -9 \\ -6 & -12 & -10 \\ -8 & 4 & -15 \end{bmatrix} = \overbrace{\begin{bmatrix} -113 & 84 & -87 \\ -112 & -7 & 84 \\ -216 & 120 & -35 \\ -260 & 120 & -28 \\ 1 & -26 & 24 \end{bmatrix}}^{5 \times 3}$$

- Applying ReLU

$$h_n = \max(0, W^T h_{n-1} + b) = \begin{bmatrix} 0 & 84 & 0 \\ 0 & 0 & 84 \\ 0 & 120 & 0 \\ 0 & 120 & 0 \\ 1 & 0 & 24 \end{bmatrix}$$

Other Common Activation Functions

- Sigmoid

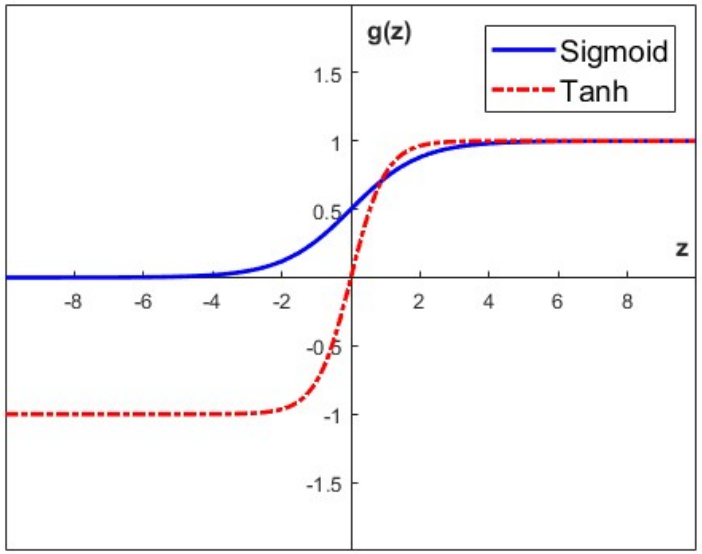
$$g(z) = \frac{1}{1+e^{-z}}$$

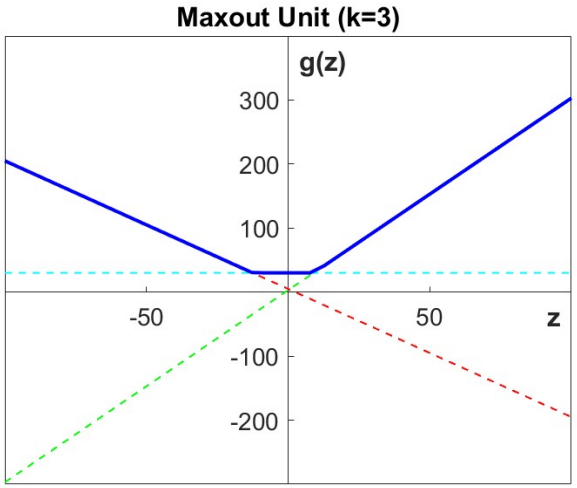
- Tanh

$$g(z) = \tanh(z)$$

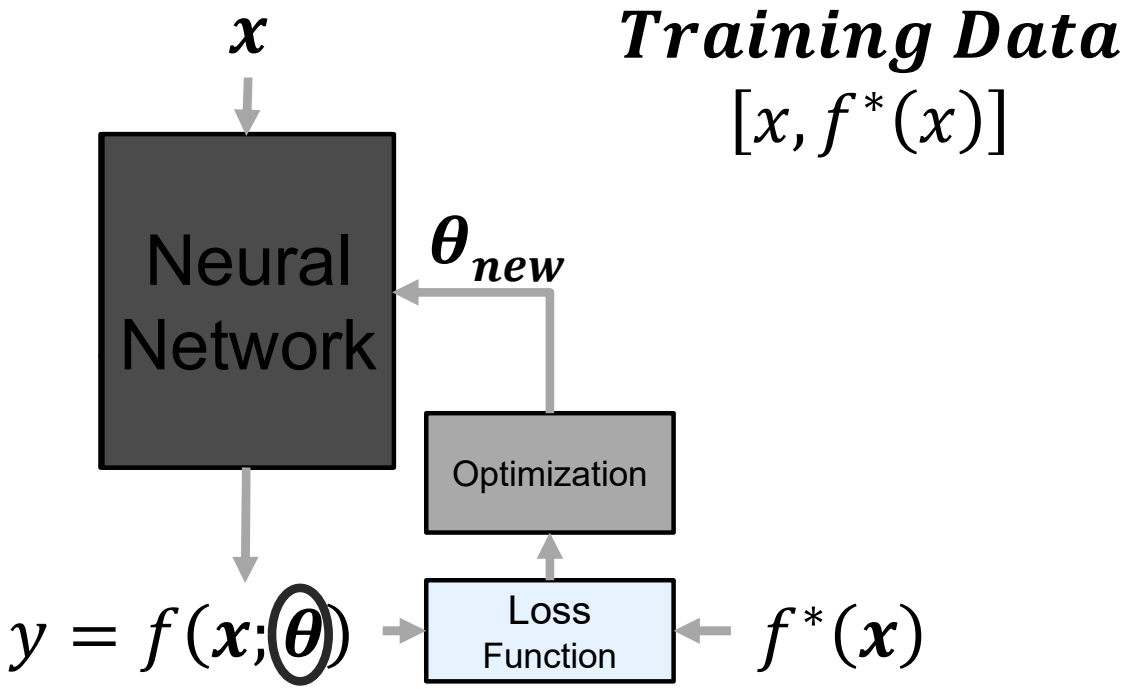
- Maxout

$$g(z) = \max\left(w_1^T z + b_1, w_2^T z + b_2, 0\right)$$





Training Artificial Neural Networks



Neural Network Loss Functions

- Thousands of training example pairs $[\boldsymbol{x}, f^*(\boldsymbol{x})]$
- The Loss function computed over all N training examples is termed the Training Loss and can be written as:

$$J(\theta) = \frac{1}{N}\sum_{i=1}^{N} L(f(x_i, \theta), f^*(x_i))$$

- The gradient of the training loss with respect to the parameters $\boldsymbol{\theta}$ can be written as:

$$\nabla_\theta J(\theta) = \nabla_\theta \left[\frac{1}{N}\sum_{i=1}^{N} L(f(x_i, \theta), f^*(x_i))\right] = \frac{1}{N}\sum_{i=1}^{N} \nabla_\theta L(f(x_i, \theta), f^*(x_i))$$

Backpropagation

- Question 1:
  How can we compute the gradients of the parameters of an arbitrary neural network?

- Question 2:
  How can we make the gradient computation efficient?

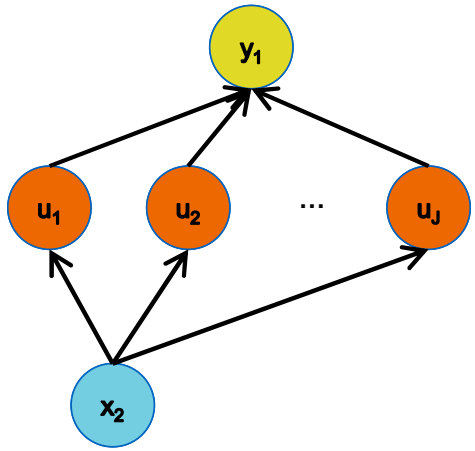
Chain Rule

- Given:

$$\boldsymbol{y} = g(\boldsymbol{u}) \text{ and } \boldsymbol{u} = h(\boldsymbol{x})$$

- Chain Rule:

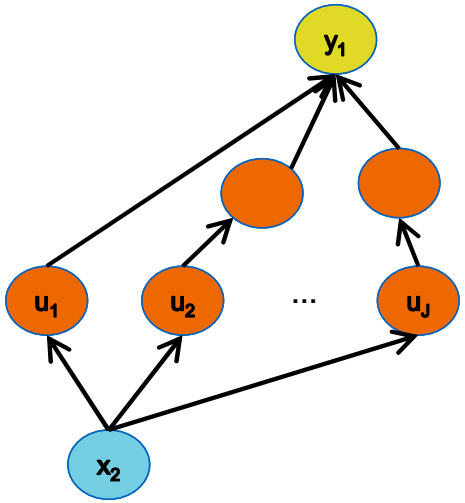
$$\frac{dy_i}{dx_k} = \sum_{j=1}^{J} \frac{dy_i}{du_j} \frac{du_j}{dx_k}, \quad \forall i, k$$



- Backpropagation is just repeated application of the chain rule from Calculus 101.

Backpropagation (Chain Rule)

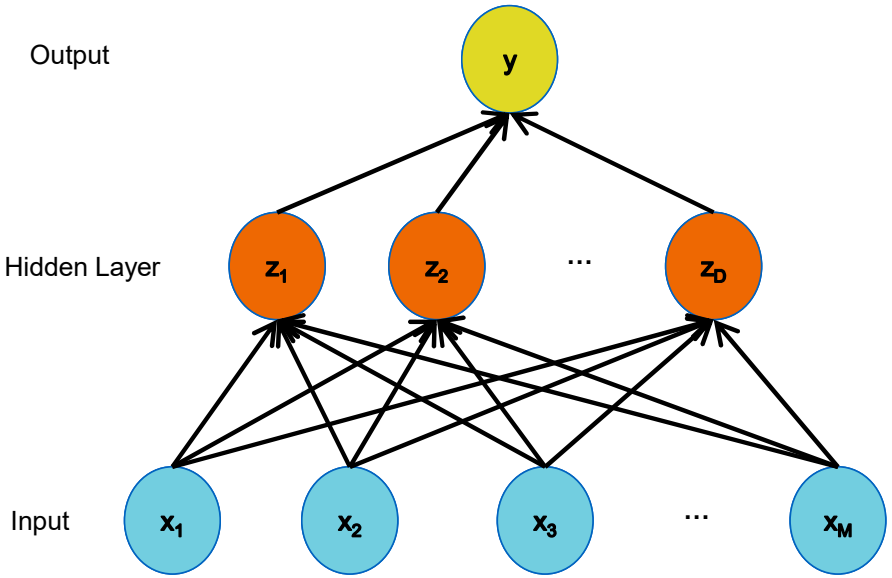
- Backpropagation:
  1. **Instantiate the computation as a directed acyclic graph**, where each intermediate quantity is a node
  2. At each node, store
     1. the intermediate quantity computed in the forward pass and
     2. the **partial derivative** of the goal with respect to that node's intermediate quantity.
  3. **Initialize** all partial derivatives to 0.
  4. Visit each node in **reverse topological order**. At each node, add its contribution to the partial derivatives of its parents

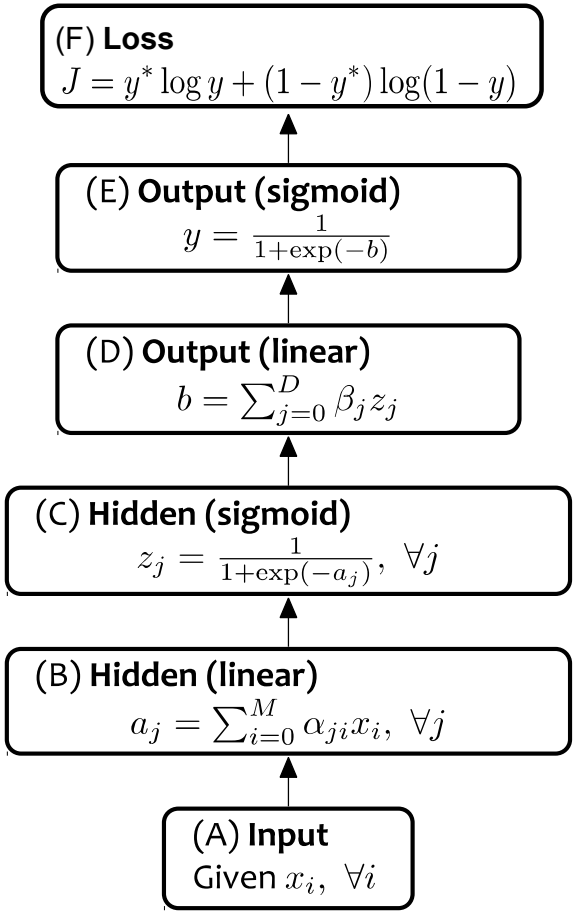


- This algorithm is also called **automatic differentiation in the reverse-mode**

Example of Backpropagation

- Binary classifier





Example of Backpropagation

Forward

$$J = y^* \log y + (1 - y^*) \log(1 - y)$$

$$y = \frac{1}{1 + \exp(-b)}$$

$$b = \sum_{j=0}^{D} \beta_j z_j$$

$$z_j = \frac{1}{1 + \exp(-a_j)}$$

$$a_j = \sum_{i=0}^{M} \alpha_{ji} x_i$$

Backward

$$\frac{dJ}{dy} = \frac{y^*}{y} + \frac{(1 - y^*)}{y - 1}$$

$$\frac{dJ}{db} = \frac{dJ}{dy}\frac{dy}{db}, \frac{dy}{db} = \frac{\exp(-b)}{(\exp(-b) + 1)^2}$$

$$\frac{dJ}{d\beta_j} = \frac{dJ}{db}\frac{db}{d\beta_j}, \frac{db}{d\beta_j} = z_j$$

$$\frac{dJ}{dz_j} = \frac{dJ}{db}\frac{db}{dz_j}, \frac{db}{dz_j} = \beta_j$$

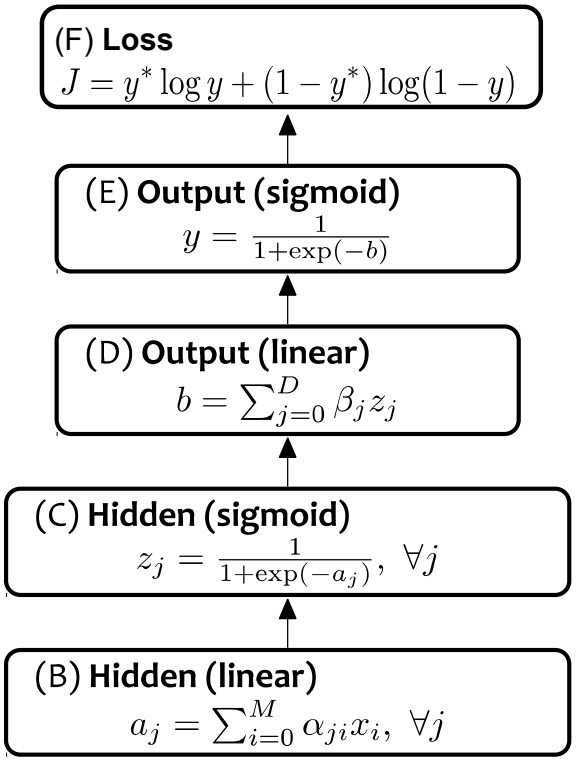
$$\frac{dJ}{da_j} = \frac{dJ}{dz_j}\frac{dz_j}{da_j}, \frac{dz_j}{da_j} = \frac{\exp(-a_j)}{(\exp(-a_j) + 1)^2}$$

$$\frac{dJ}{d\alpha_{ji}} = \frac{dJ}{da_j}\frac{da_j}{d\alpha_{ji}}, \frac{da_j}{d\alpha_{ji}} = x_i$$

$$\frac{dJ}{dx_i} = \frac{dJ}{da_j}\frac{da_j}{dx_i}, \frac{da_j}{dx_i} = \sum_{j=0}^{D} \alpha_{ji}$$

Example of Backpropagation

- So backpropagation allows us to efficiently compute the gradients for each set of network parameters



$$\frac{dJ}{db} = \frac{dJ}{dy}\frac{dy}{db} = \left(\frac{y^*}{y} + \frac{(1-y^*)}{y-1}\right)\left(\frac{\exp(-b)}{(\exp(-b)+1)^2}\right)$$

$$\frac{dJ}{d\beta_j} = \frac{dJ}{dy}\frac{dy}{db}\frac{db}{d\beta_j} = \left(\frac{y^*}{y} + \frac{(1-y^*)}{y-1}\right)\left(\frac{\exp(-b)}{(\exp(-b)+1)^2}\right) z_j$$

.
.
.

Parameter Optimization

- Batch Gradient Descent is an iterative first order optimization procedure

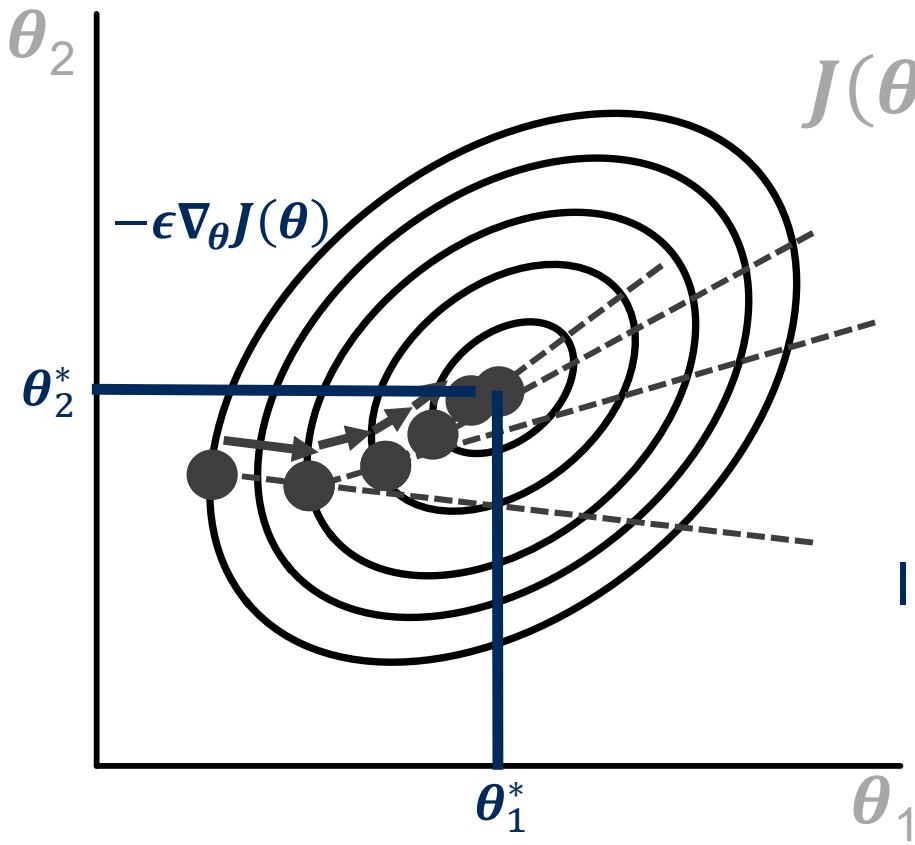
- Batch Gradient Descent Algorithm:
  - Initialize parameters $\boldsymbol{\theta}$
  - While Stopping Condition is Not Met:
    - Compute gradient of loss function over all training examples:

$$\nabla_\theta J(\theta) = \frac{1}{N}\sum_{i=1}^{N} \nabla_\theta \, L[f(x_i, \theta), f^*(x_i)]$$

    - Update parameters according to:

$$\theta \leftarrow \theta - \epsilon \nabla_\theta J(\theta)$$

Batch Gradient Descent



Initialize parameters θ

Parameter Initialization and Stopping Conditions

- Parameter Initialization:
  - Weights: initialized by randomly sampling from a standard normal distribution
  - Biases: initialized to 0
  - Many other heuristics exist

- Stopping Conditions:
  - Number of iterations: How many training iterations the neural network has performed
  - Change In $\boldsymbol{\theta}$ value: Stop if $\boldsymbol{\theta}_{new} - \boldsymbol{\theta}_{old} <$ Threshold
  - Change In $\boldsymbol{J}(\boldsymbol{\theta})$ value: Stop if $J(\boldsymbol{\theta}_{new}) - J(\boldsymbol{\theta}_{old}) <$ Threshold

Batch Gradient Descent

- Backpropagation used to compute $\boldsymbol{\nabla_\theta J(\theta)}$ is very expensive to compute over the whole training dataset.

$$\boldsymbol{\nabla_\theta J(\theta)} = \frac{1}{\mathrm{N}} \sum_i^N \boldsymbol{\nabla_\theta} L\,[f_i(x, \theta), f_i^*(x)] \text{ is a mean!}$$

  - Standard error of the mean estimated from N samples is $\frac{\sigma}{\sqrt{N}}$, where σ is the standard deviation of the value of the samples
  - Using all samples to estimate the gradient results in less than linear return in accuracy of this estimate

- Instead, we can use a small subsample (Minibatch) of the training data to estimate the gradient

Stochastic (minibatch) Gradient Descent

- Stochastic (minibatch) Gradient Descent Algorithm:
  - Initialize parameters $\boldsymbol{\theta}$
  - While Stopping Condition is False:
    - Sample a preset number N' of examples (minibatch) from the training data
    - Compute gradient of Loss Function Over all training examples:

$$\nabla_\theta J(\theta) = \frac{1}{N'} \sum_i^{N'} \nabla_\theta L\left[f_i(x,\theta), f_i^*(x)\right]$$

    - Update parameters according to:

$$\theta \leftarrow \theta - \epsilon \nabla_\theta J(\theta)$$

What Minibatch Size To Use ?

- GPUs work better with powers of 2 batch sizes
- Large batch sizes > 256:
  - Hardware underutilized with very small batch sizes.
  - More accurate estimate of the gradient, but with less than linear returns
- Small batch size < 64
  - Small batches can offer a regularizing effect. The best generalization error is often achieved with batch size of 1.
  - Small batch sizes allow for faster convergence, as the algorithm can compute the parameter updates rapidly

- Always make sure dataset is shuffled before sampling minibatch

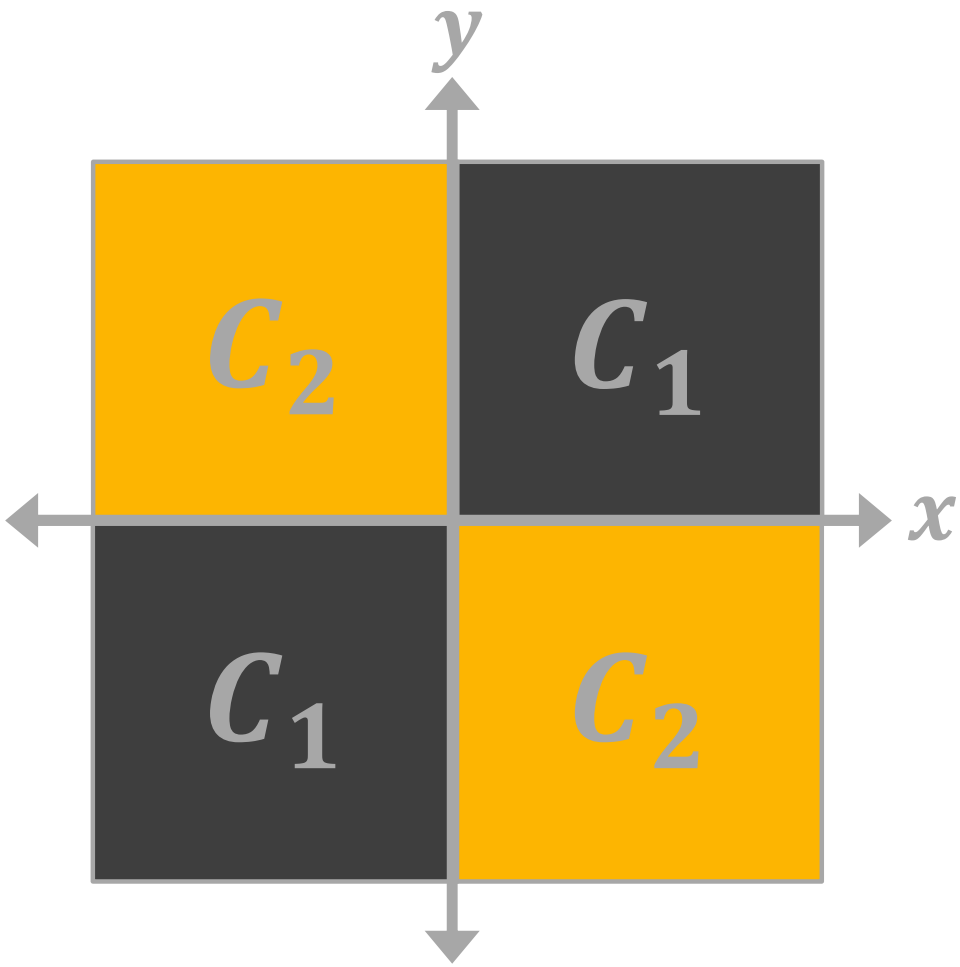
SGD Variations

- Many variations of SGD exist
  - Momentum SGD, Nesterov Momentum SGD
  - Ada-Grad, RMS-Prop
  - ADAM (Adaptive Moment Estimation)

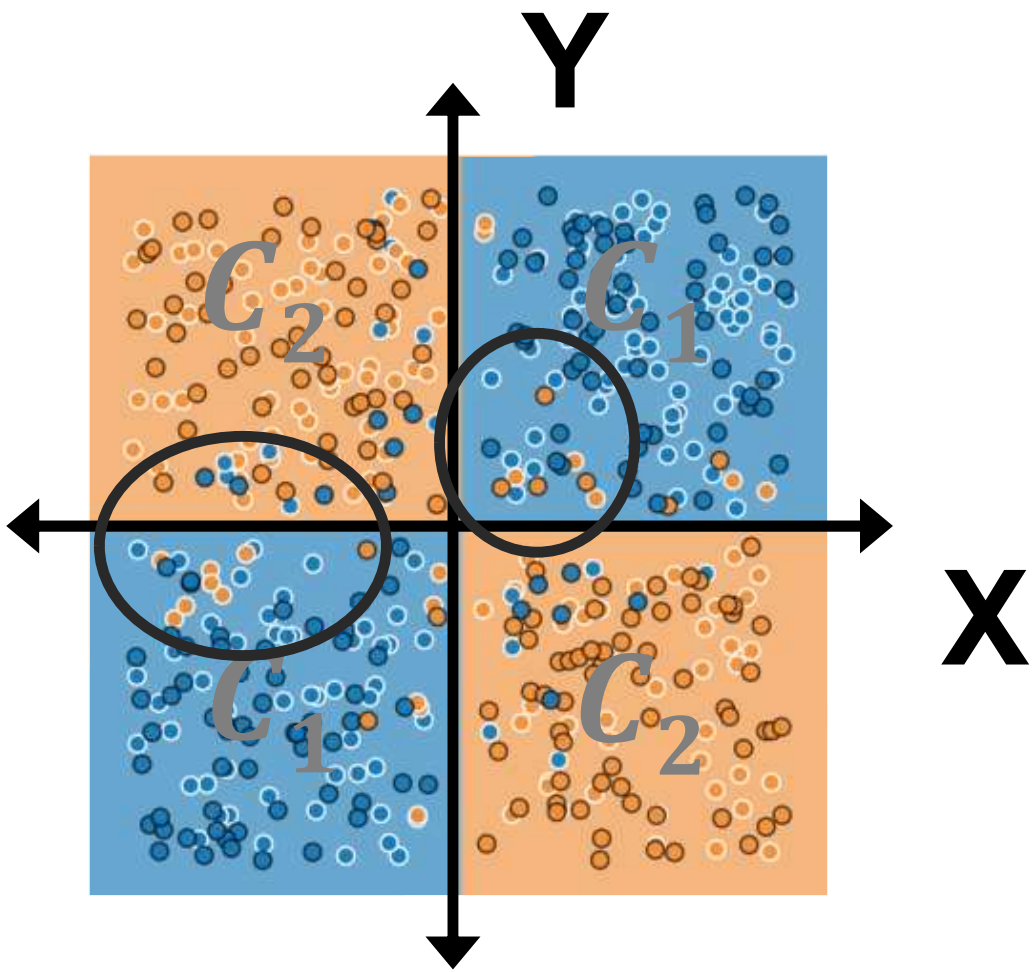
- Which one to use ?
  - Usually ADAM: Implemented in most deep neural network libraries, fairly robust to the choice of the learning rate and other hyperparameters

- http://deeplearningbook.org

Toy Example - XOR



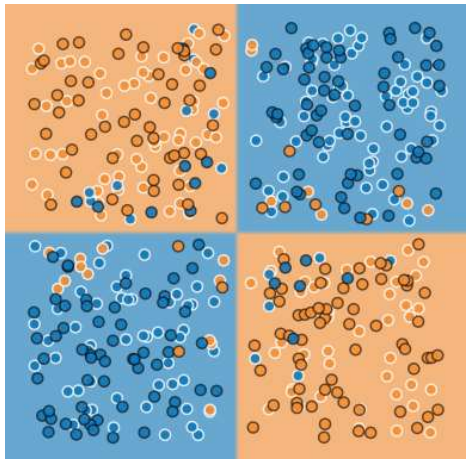
Toy Example - XOR

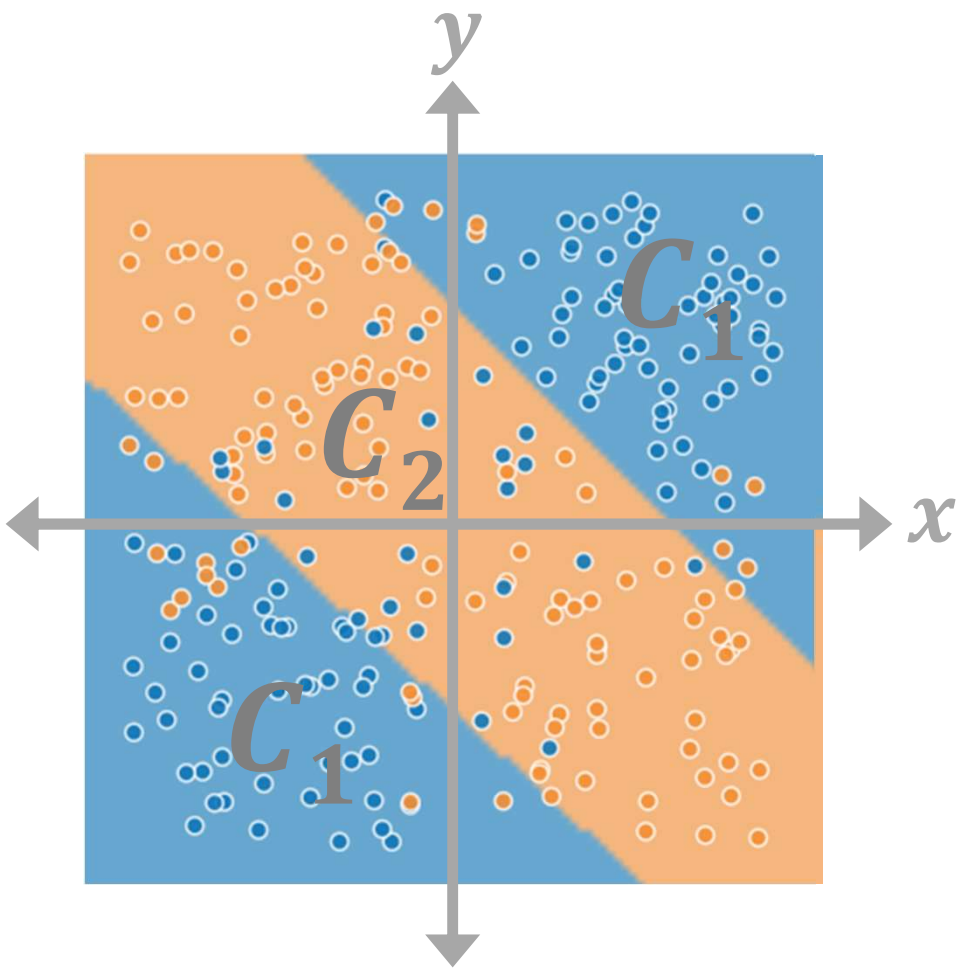


http://playground.tensorflow.org

Toy Example

- Initial Design:
  - 1 Layer NN, 2 Hidden Units/Layer
  - Train set Loss: 0.264
  - Val set Loss: 0.268
  - Minimum Loss achievable: 0.1

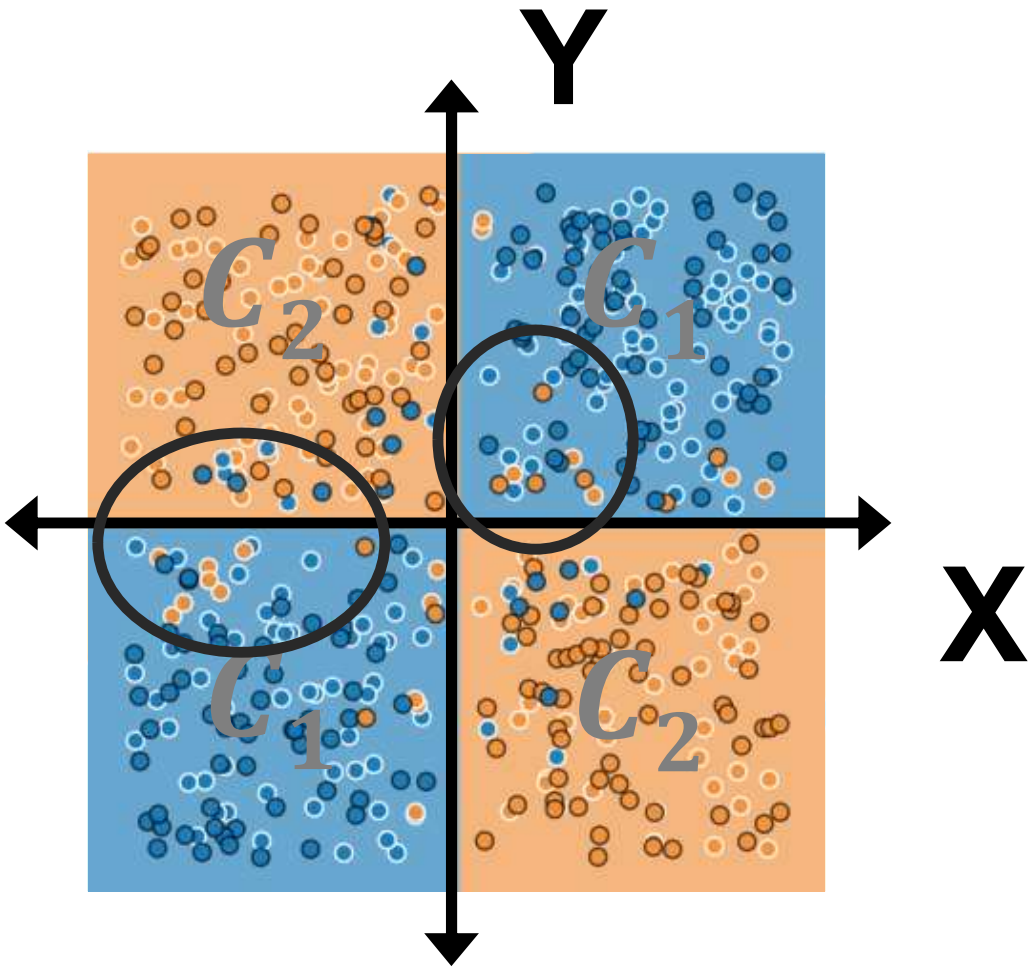




http://playground.tensorflow.org

Toy Example

- New Design:
  - 6 Layer NN, 6 Hidden Units/Layer
  - Train set Loss: 0.1
  - Val set Loss: 0.45
  - Minimum Loss achievable: 0.1



http://playground.tensorflow.org

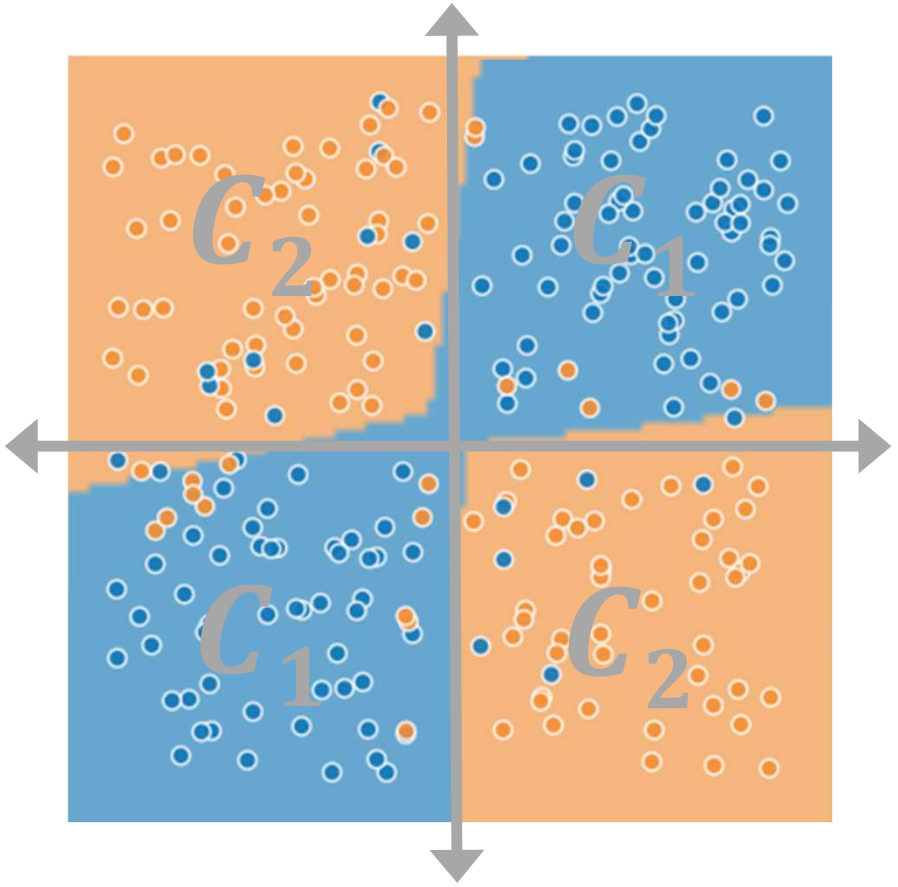
Regularization - Parameter Norm Penalties

- $J(\theta)_{reg} = J(\theta) + \alpha\Omega(\theta)$
- α is a hyperparameter that weights the relative contribution of the norm penalty to the value of the loss function
- $\Omega(\boldsymbol{\theta})$ is a measure of how large $\boldsymbol{\theta}$'s value is, usually an Lp Norm
- We usually only constrain the size of weights and not biases
- $J(\theta)_{reg} = J(\theta) + \alpha\Omega(W)$

L2-Norm Parameter Penalty

$$\Omega(W) = \frac{1}{2} W^T W = \frac{1}{2} \|W\|_2^2$$

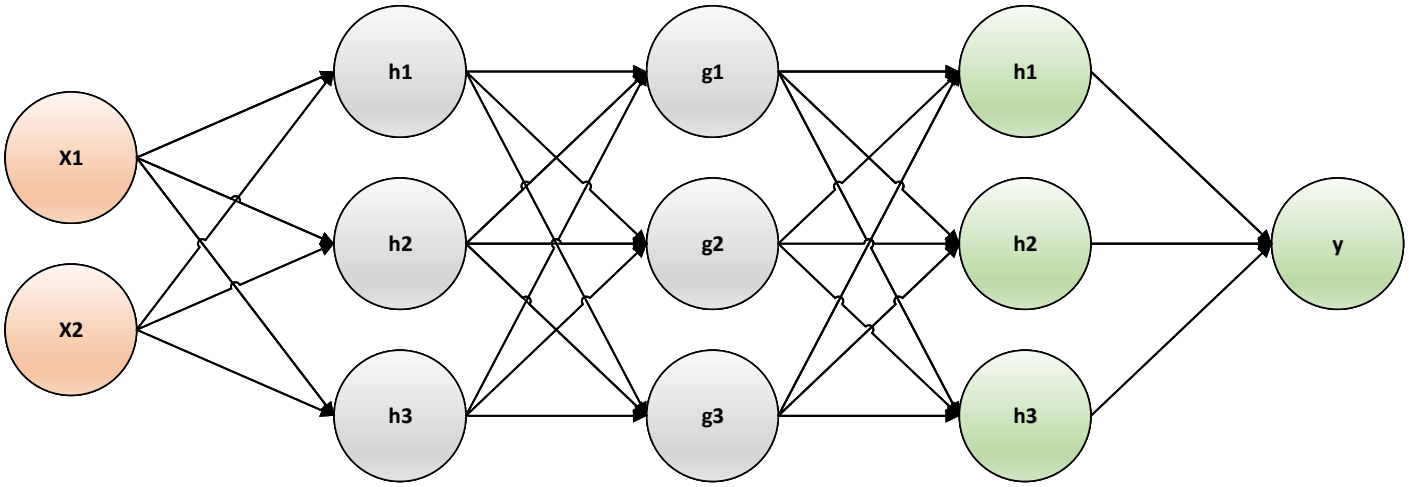
- New Design:
  - 6 Layer NN, 6 Hidden Units/Layer
  - Minimum Loss achievable: 0.1
  - L2-Norm Penalty
  - Train set Loss: 0.1 0.176
  - Val set Loss: 0.45 0.182



http://playground.tensorflow.org

Dropout

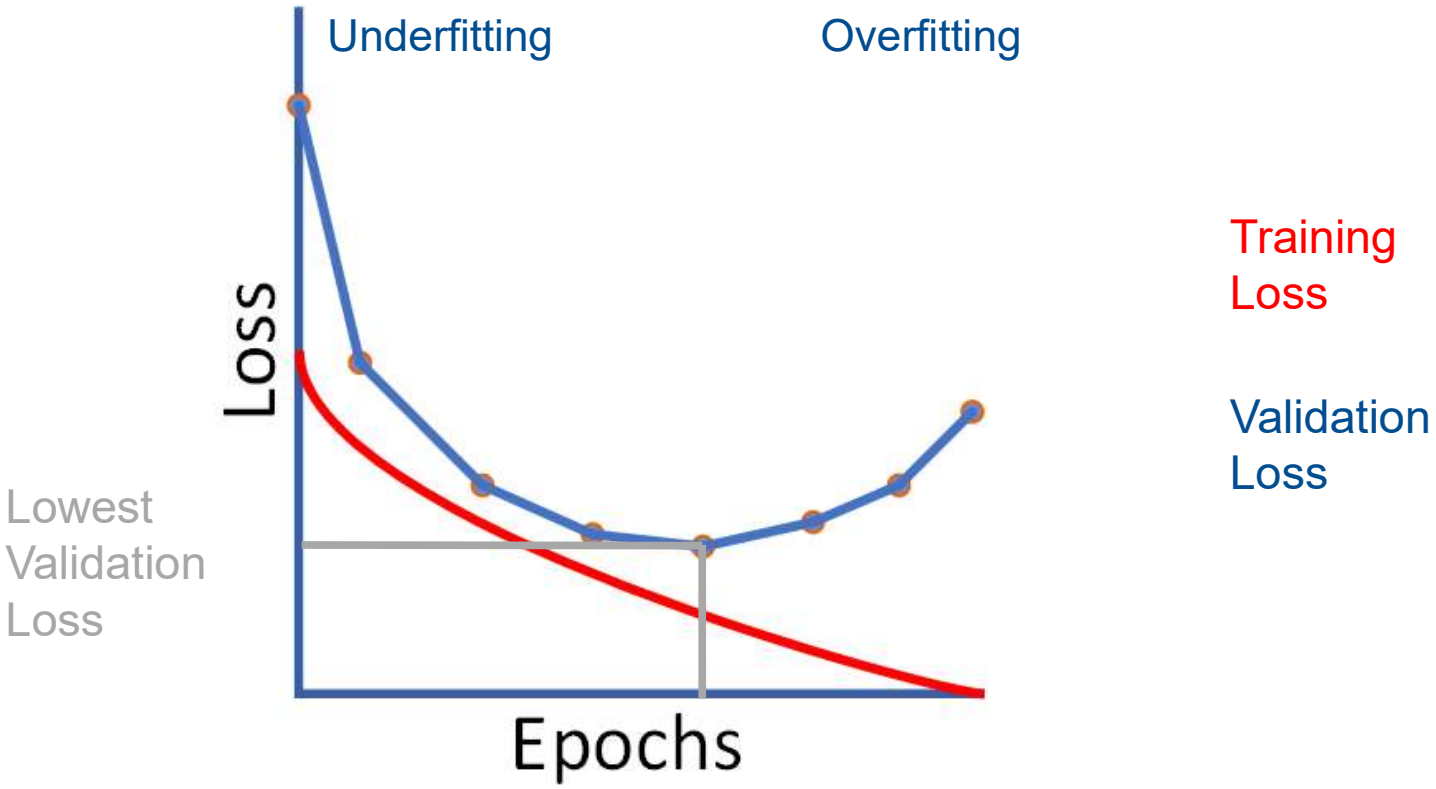
$$P_{keep} = 0.5$$



Multiply **Weights** by P_{keep} at the end of training

- Computationally inexpensive but powerful regularization method
- Does not significantly limit the type of model or training procedure that can be used
- Dropout layers are practically implemented in all neural network libraries

Early Stopping



Outline

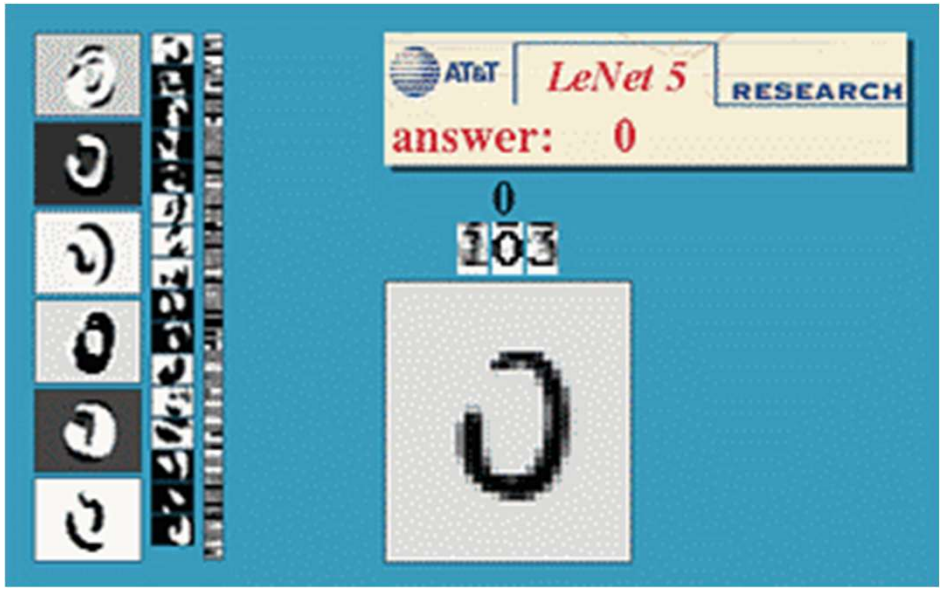
- Machine Learning
- Neural networks
- **Convolutional Neural Networks**
- Recurrent Neural Networks
- Transformers

CNN History

- First proposed by Kunihiko Fukushima in 1980, the “neocognitron” included the two main aspects of modern CNN, convolution and max pooling layers.

- Alex Waibel then applied shared convolution weights and backprop to the Temporal Convolution Neural Network in 1987.

- In 1989, Yann LeCunn improved on these works and applied backprop to CNNs on image data, beating hand engineered performance on the MNIST character recognition dataset in 1989.



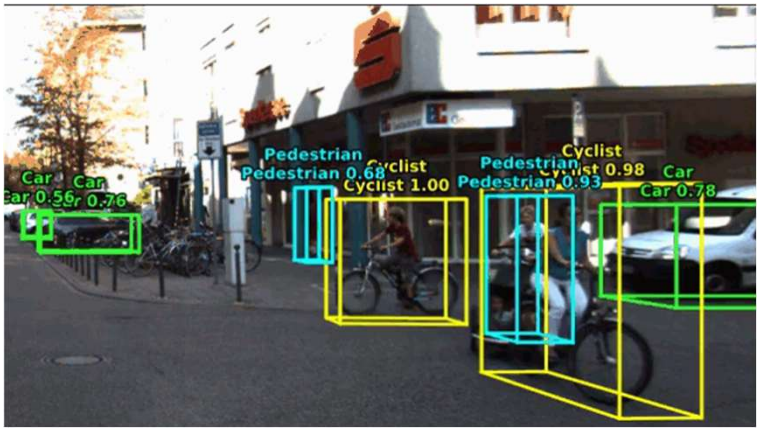
Convolutional Neural Networks

IMAGENET

- 1,000 object classes (categories).
- Images:
  - 1.2 M train
  - 100k test.



4



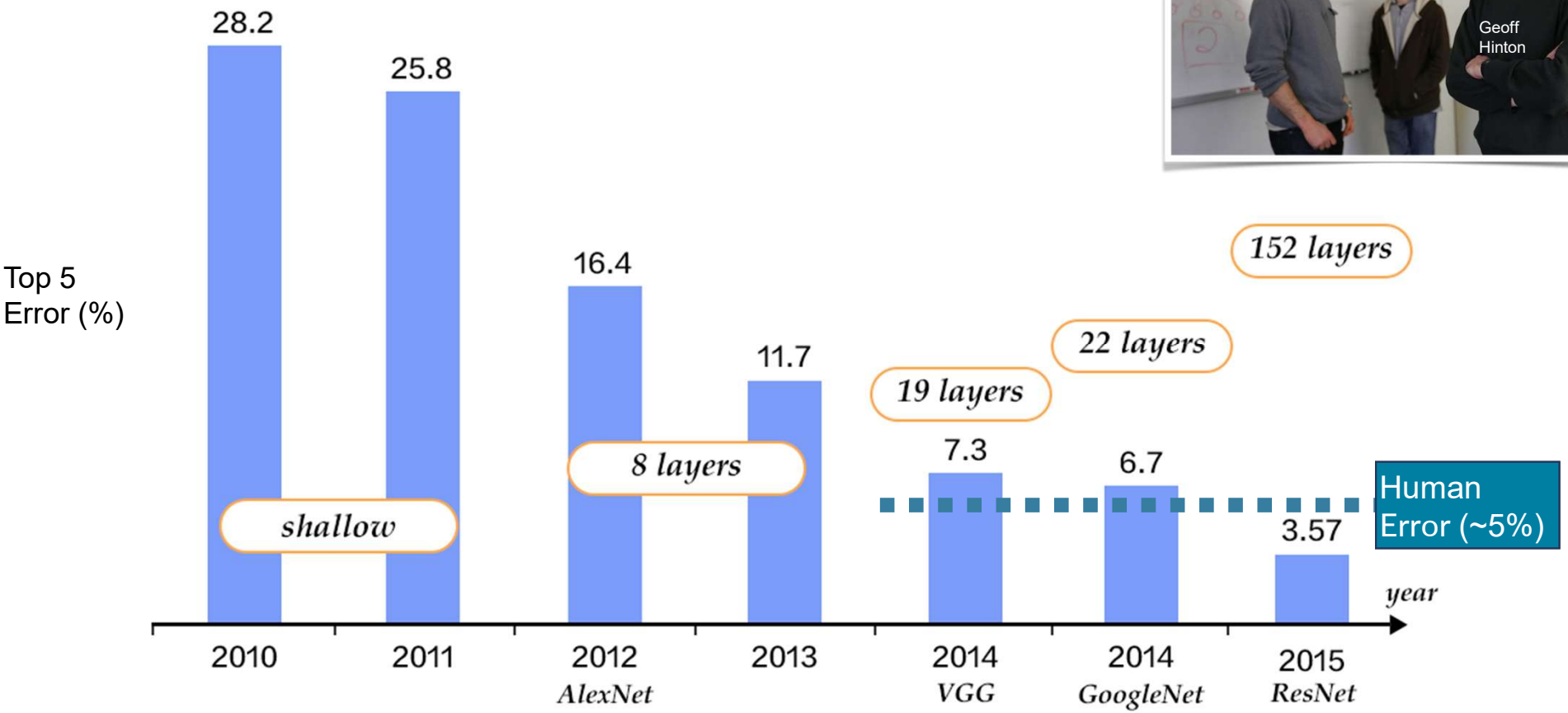
Code at: https://github.com/kujason/avod



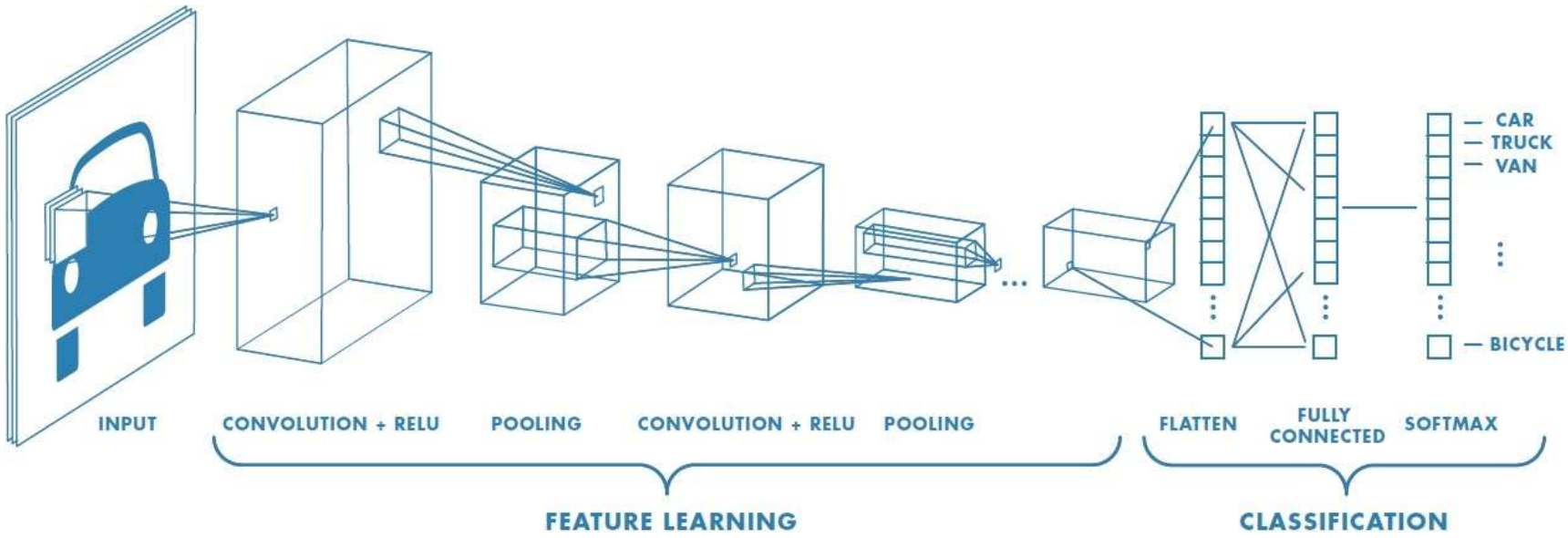
Code at: https://github.com/oandrienko/fast-semantic-segmentation

ImageNet Large Scale Visual Recognition Challenge (ILSVRC)





Convolutional Neural Networks

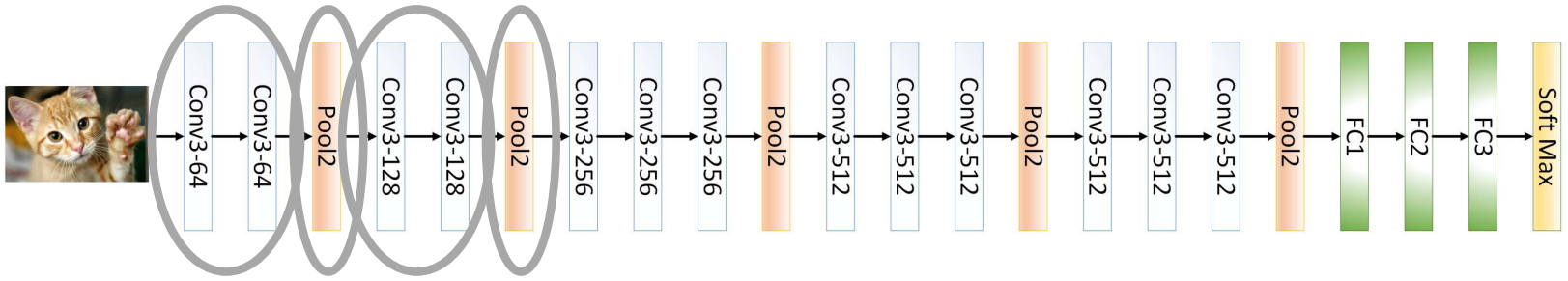


Three types of operations:

1. Convolution
2. Pooling
3. Non-Linearity (e.g, ReLU)

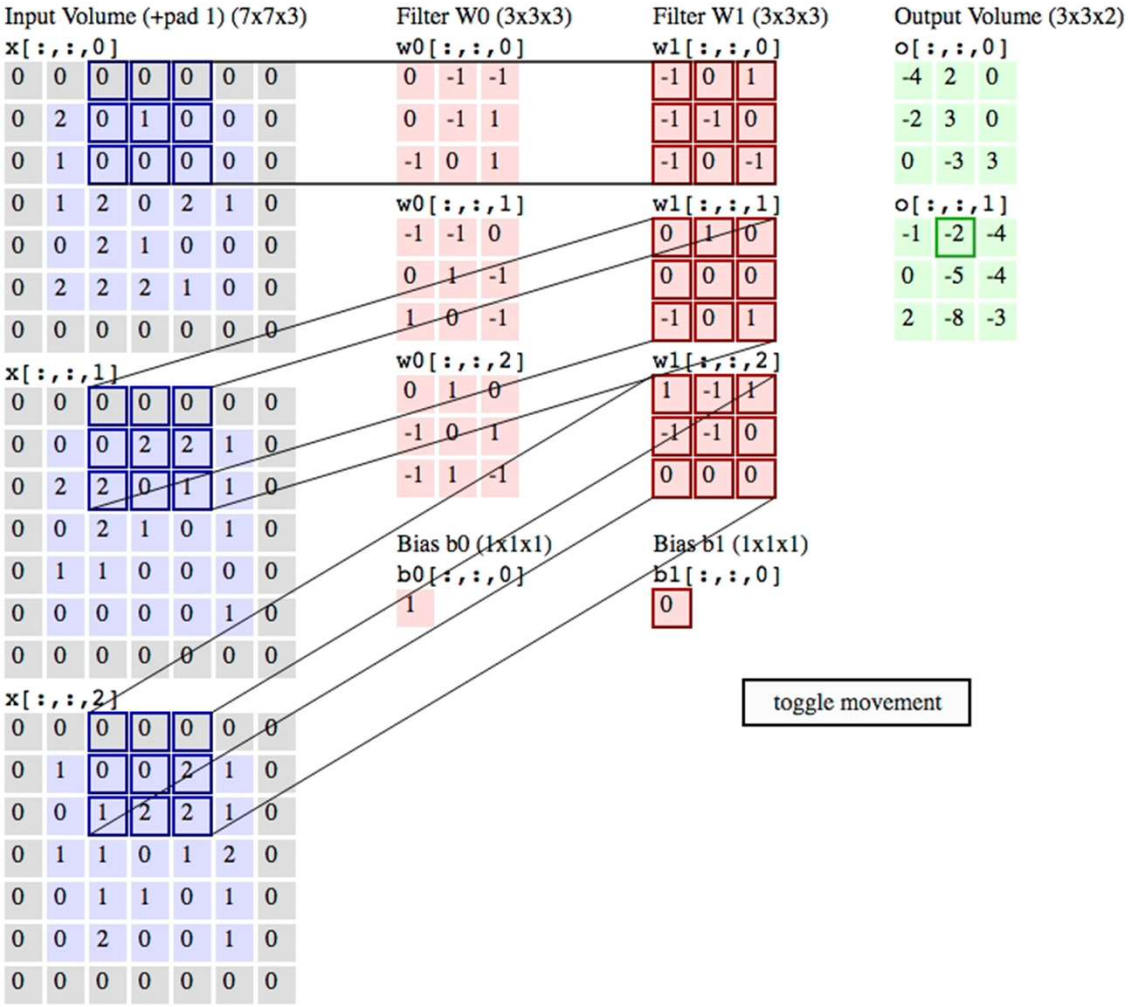
Convolutional Neural Networks

- Used for processing data defined on grid
- 1D time series data, 2D images, 3D videos and voxel grids
- Two major type of layers:
  - Convolution Layers
  - Pooling Layers
- Example: VGG-16



Convolutions

- A window of weights called a ‘kernel’ or ‘filter’ is convolved with an input image (here we see 5x5x3 input image)
- This results in significantly less parameters than if we were to connect each pixel to a weight!
- Each kernel can learn to pick out specific features like edges

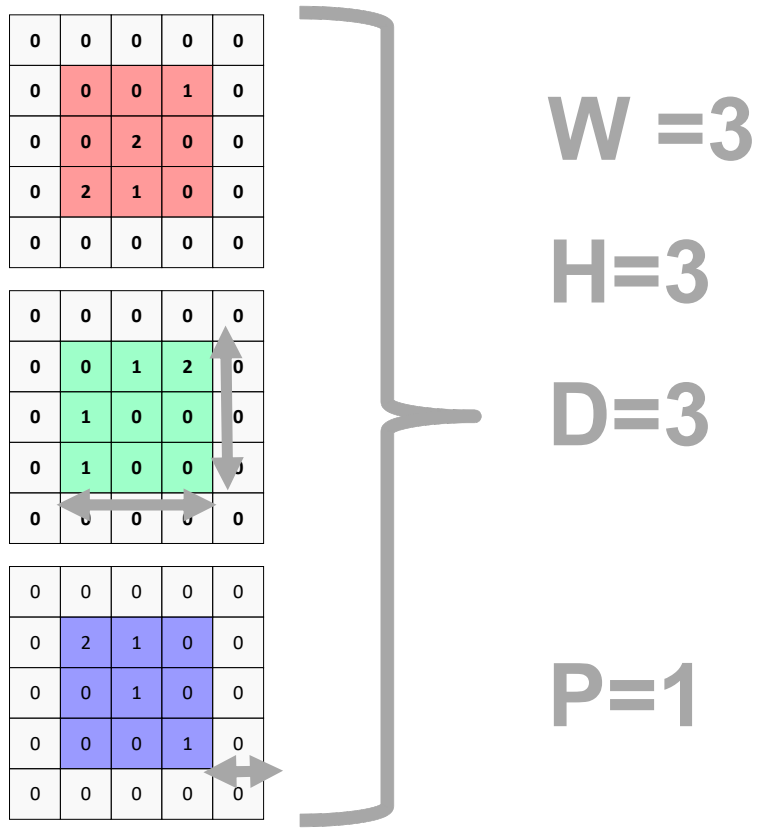


http://cs231n.github.io/convolutional-networks/

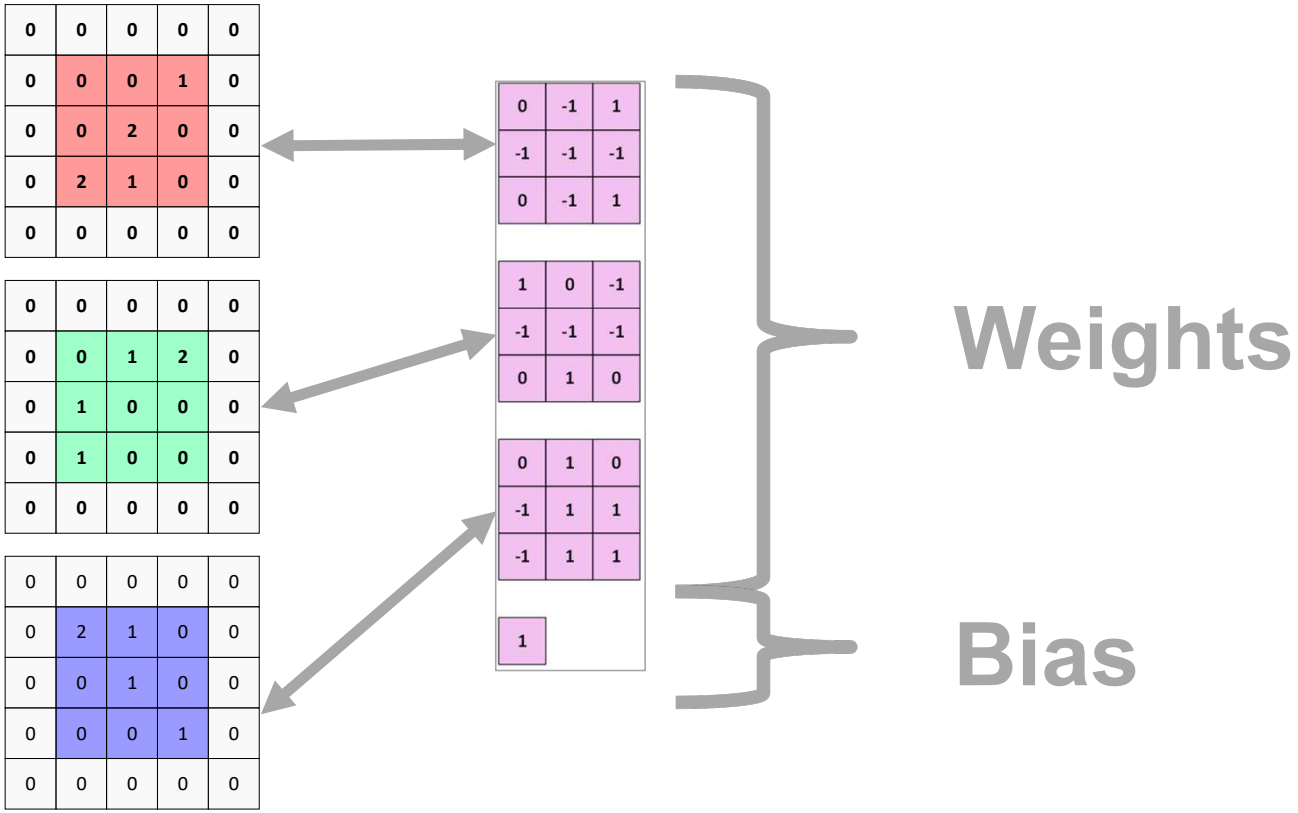
Convolution (Cross Correlation)

- Width: horizontal dimension of input volume
- Height: vertical dimension of input volume
- Depth: number of channels of input volume

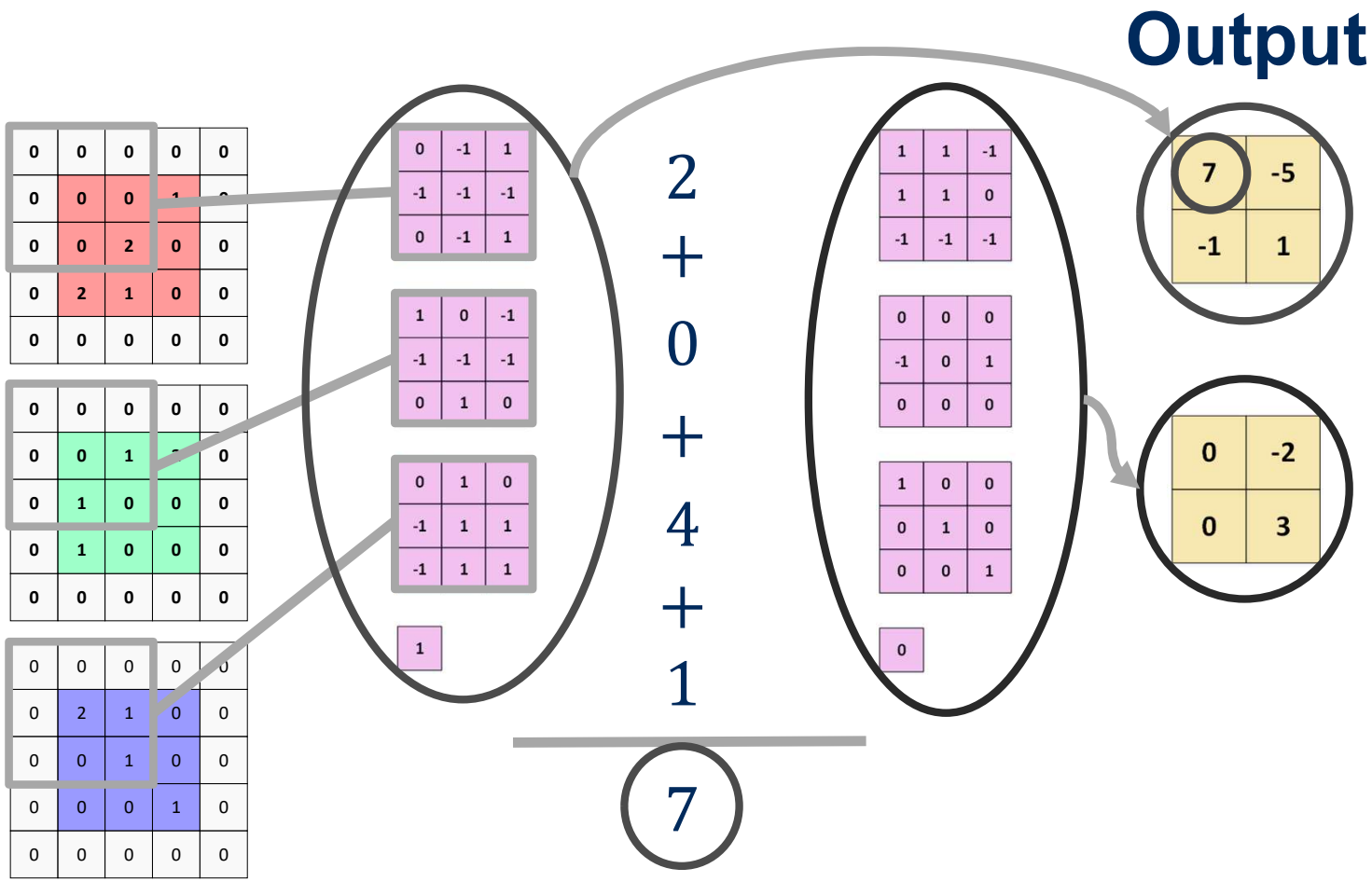
- Padding size: essential to retain shape!



Convolution (Cross Correlation)

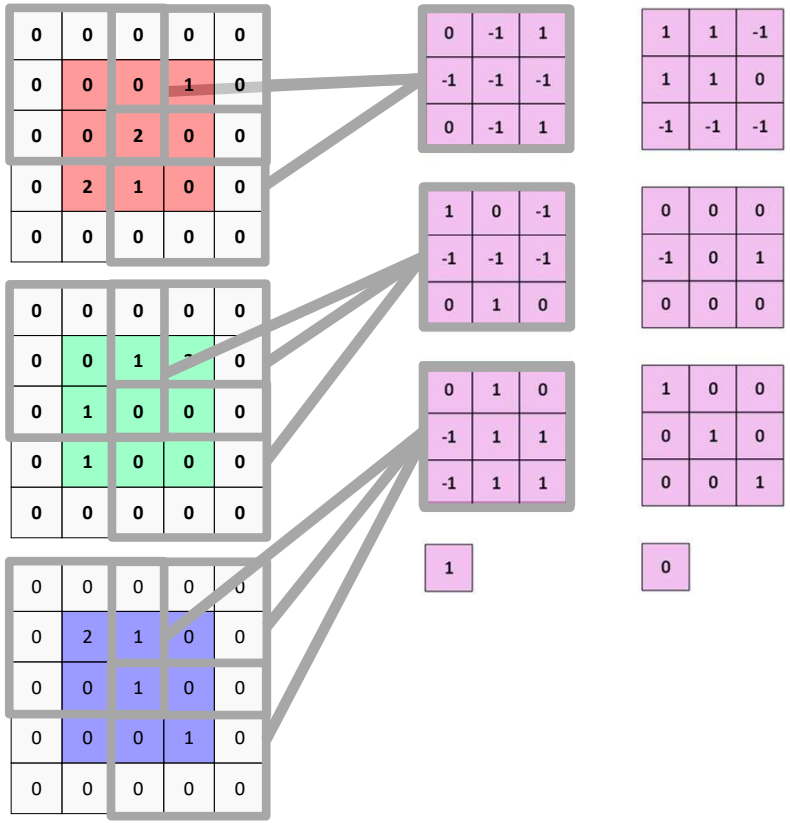


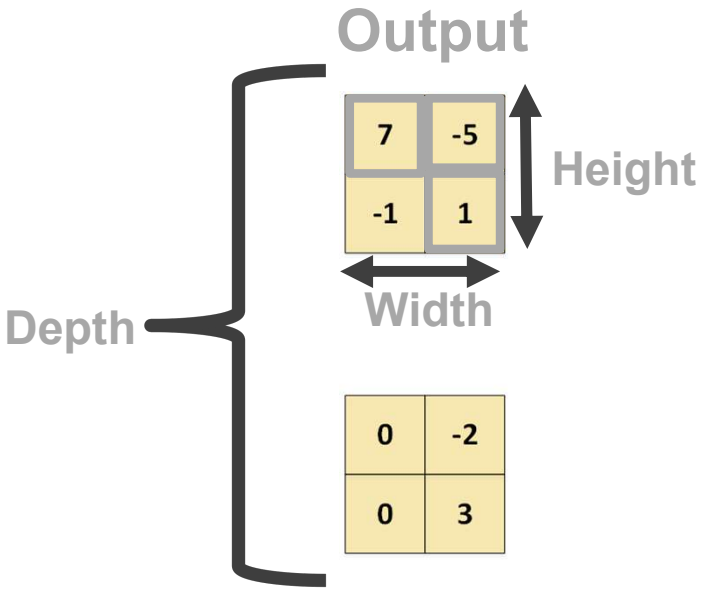
Convolution (Cross Correlation)



Cross Correlation

- **Stride:** The number of pixels the filter is moved in the vertical and horizontal direction.





Output Volume Shape

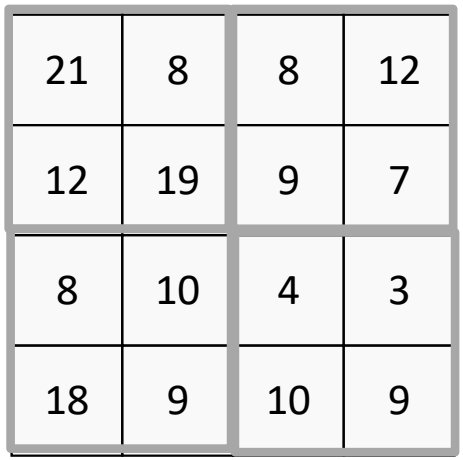
- Filters are size $\boldsymbol{m \times m}$
- Number of filters = $\boldsymbol{K}$
- Stride = $\boldsymbol{S}$, Padding = $\boldsymbol{P}$

$$\boldsymbol{W_{out}} = \frac{\boldsymbol{W_{in}} - m + 2 \times P}{S} + 1$$

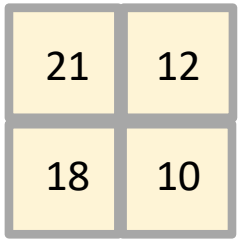
$$\boldsymbol{H_{out}} = \frac{\boldsymbol{H_{in}} - m + 2 \times P}{S} + 1$$

$$\boldsymbol{D_{out}} = K$$

Pooling Layers: Max Pooling



$$\max(21, 8, 12, 19) = 21$$



Output Volume Shape

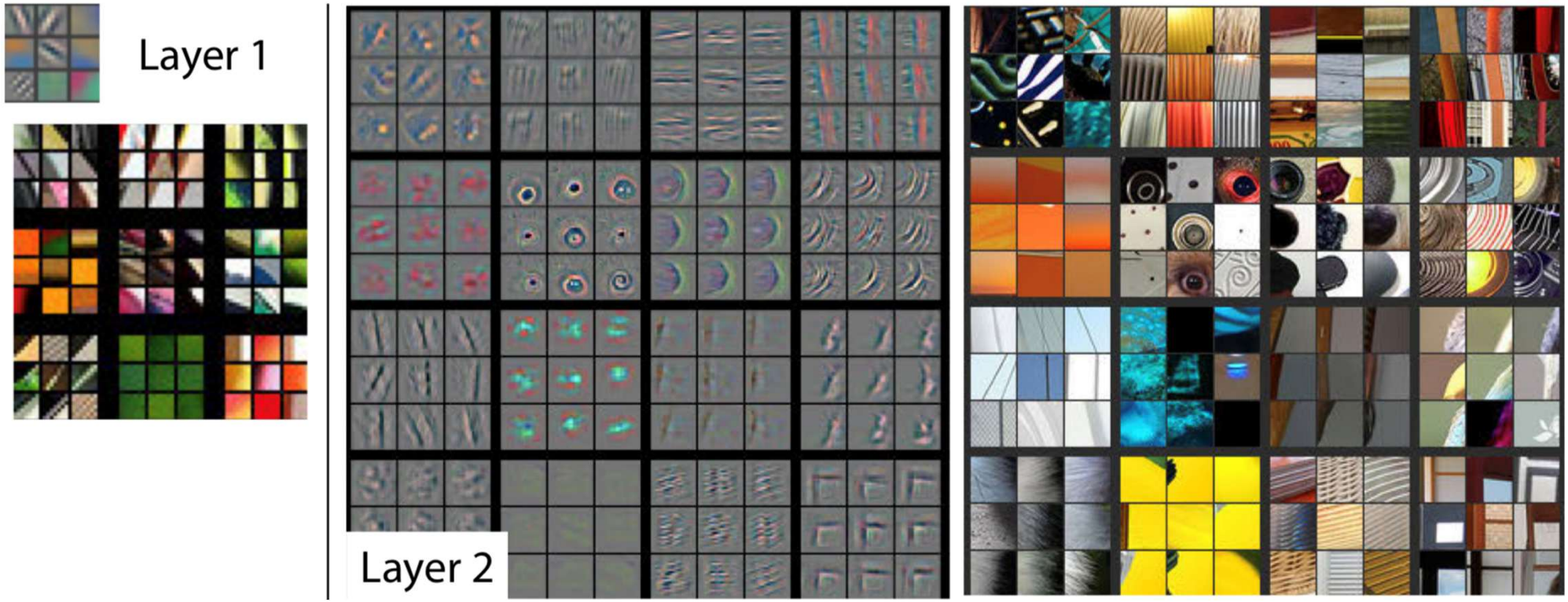
- Pool size $\boldsymbol{n} \times \boldsymbol{n}$
- Stride = $\boldsymbol{S}$

$$\boldsymbol{W_{out}} = \frac{\boldsymbol{W_{in}} - n}{S} + 1$$

$$\boldsymbol{H_{out}} = \frac{\boldsymbol{H_{in}} - n}{S} + 1$$

$$\boldsymbol{D_{out}} = \boldsymbol{D_{in}}$$

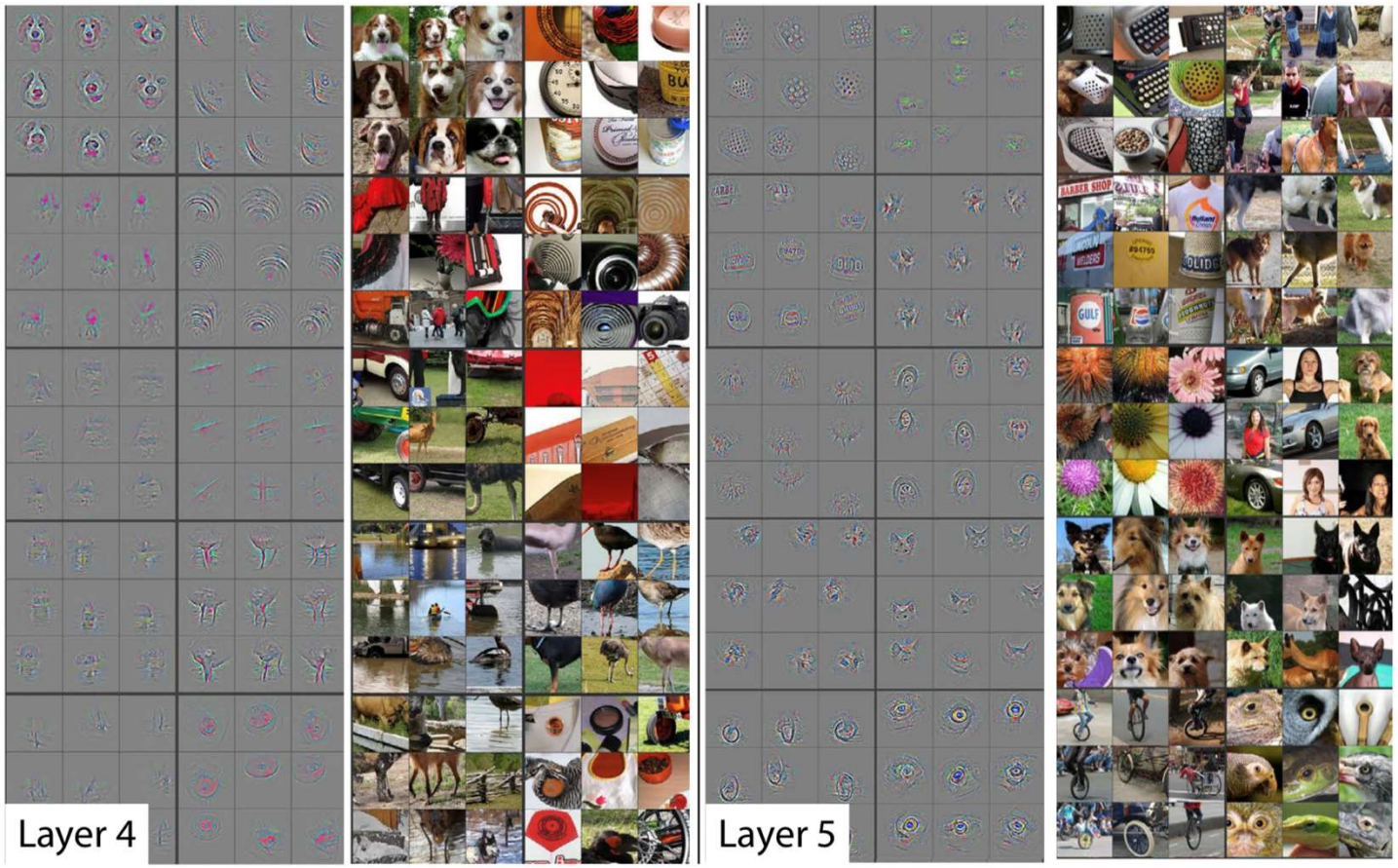
Convolutions | What do they pick out?



Lower convolutional layers pick out edges and basic shapes.

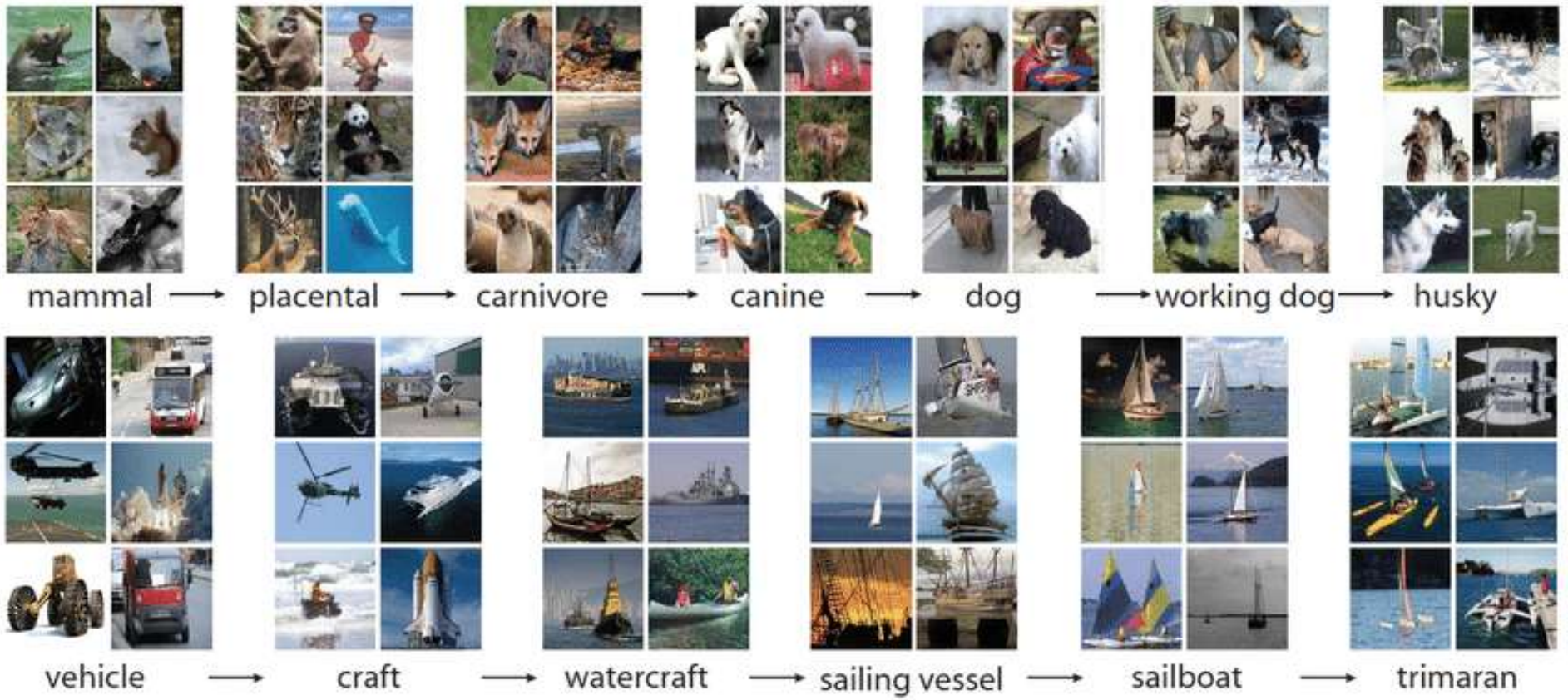
Zeiler, Matthew D., and Rob Fergus. "Visualizing and understanding convolutional networks." European conference on computer vision. Springer, Cham, 2014.

Convolutions | What do they pick out?

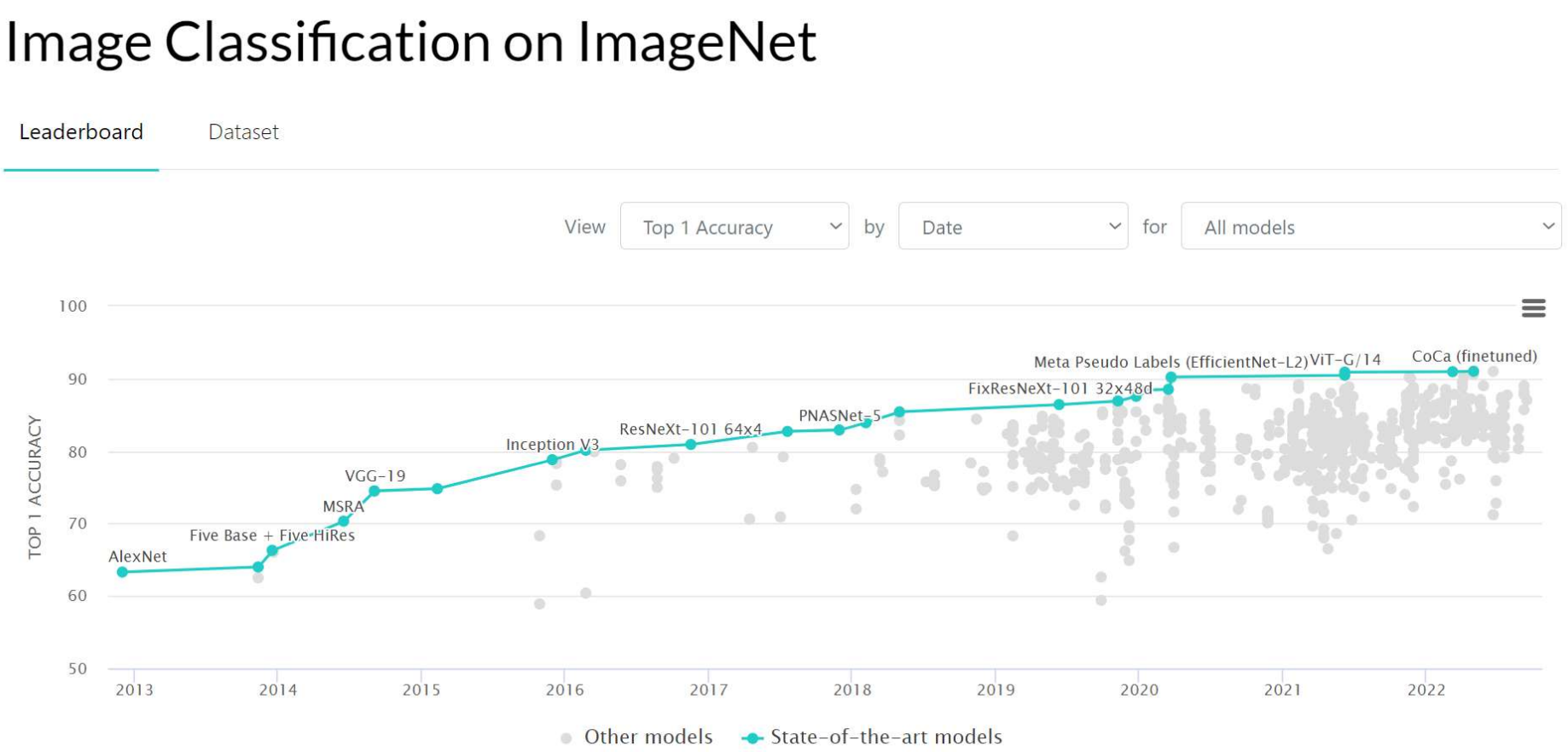


Higher layers can pick out more complicated structures.

Convnet performance evaluation - Imagenet

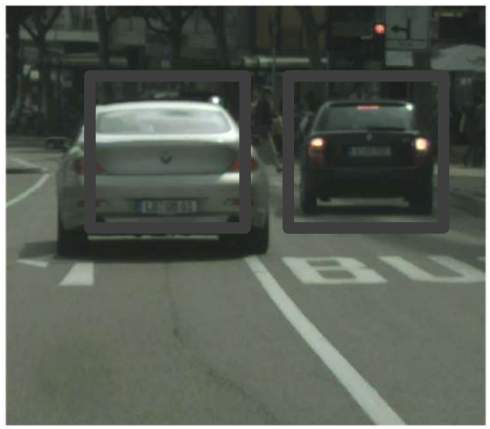


Convnet performance - Imagenet



Advantages of ConvNets

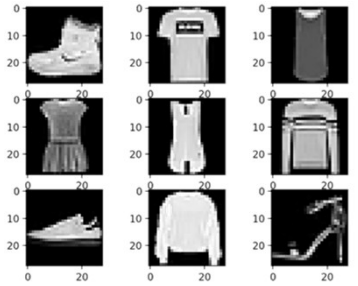
- Convolutional neural networks are a natural choice to process images by design
- Convolutional layers have fewer parameters than fully connected layers, reducing the chances of overfitting
- Convolutional layers use the same parameters to process every block of the image. Along with pooling layers, this leads to translation invariance, which is often important for image understanding tasks



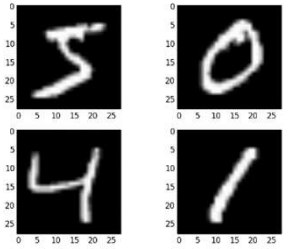
Putting it together – Fashion MNIST or MNIST Tutorials using Keras

- Fashion MNIST: 70,000 Ultra-small images of fashion items
  - Easy instructions on using Keras in python to give it a go.
    - https://www.datacamp.com/community/tutorials/convolutional-neural-networks-python
- MNIST: The original CNN dataset
  - https://machinelearningmastery.com/how-to-develop-a-convolutional-neural-network-from-scratch-for-mnist-handwritten-digit-classification/

- Challenge Problem: Achieve 94% classification accuracy on CIFAR-10 using a Resnet-18 backbone. Needs all the tricks
  - Regularization
  - Normalization
  - Data augmentations
  - Learning rate schedules, rates tuning
  - Hyperparameter search



Fashion MNIST



MNIST



CIFAR-10

Outline

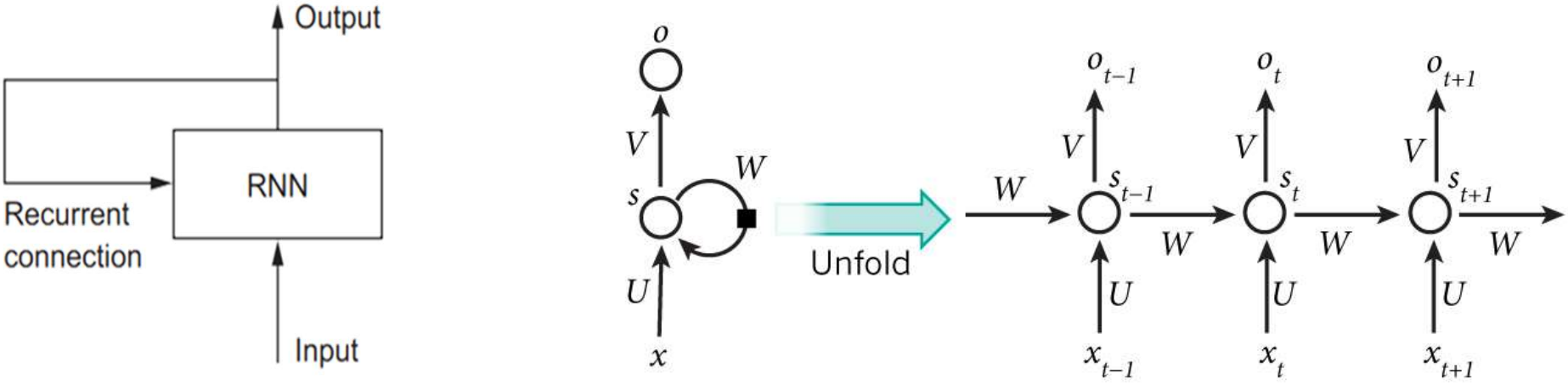
- Machine Learning
- Neural networks
- Convolutional Neural Networks
- **Recurrent Neural Networks**
- Transformers

Sequential Data (Time Series, Natural Language Processing, Object Tracking)

- Not all problems can be converted into one with fixed-length inputs and outputs
- Problems such as Speech Recognition or Time-series Prediction require a system to store and use context information
  - Simple case: Output YES if the number of 1s is even, else NO
    1000010101 – YES, 100011 – NO, …
- Hard/Impossible to choose a fixed context window
  - There can always be a new sample longer than anything seen
- Robotics often has time series problems, e.g. odometry, object tracking

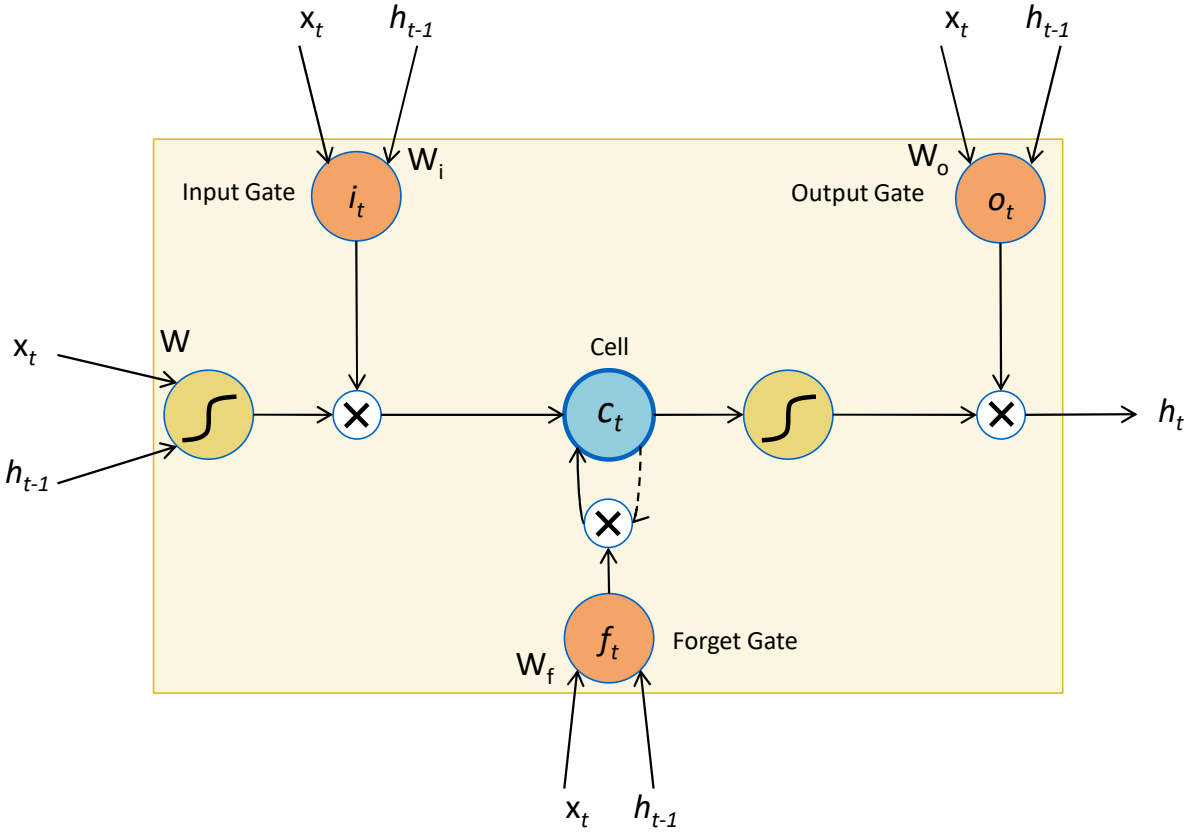
Motivation for adding recurrence

- Recurrent Neural Networks take the previous output or hidden states as inputs. The composite input at time t has some historical information about the happenings at time T < t
- RNNs are useful as their intermediate values (state) can store information about past inputs for a time that is not fixed a priori
- RNNs proved difficult to train due to gradient vanishing and exploding when backpropagating through time, and rigid in the way the system evolved, needing attention to amplify long range connections.



The LSTM Cell

- The LSTM uses this idea of "Constant Error Flow" for RNNs to create a "Constant Error Carousel" (CEC) which ensures that gradients don't decay
- The key component is a memory cell that acts like an accumulator (contains the identity relationship) over time
- Instead of computing new state as a matrix product with the old state, it rather computes the difference between them. Expressivity is the same, but gradients are better behaved



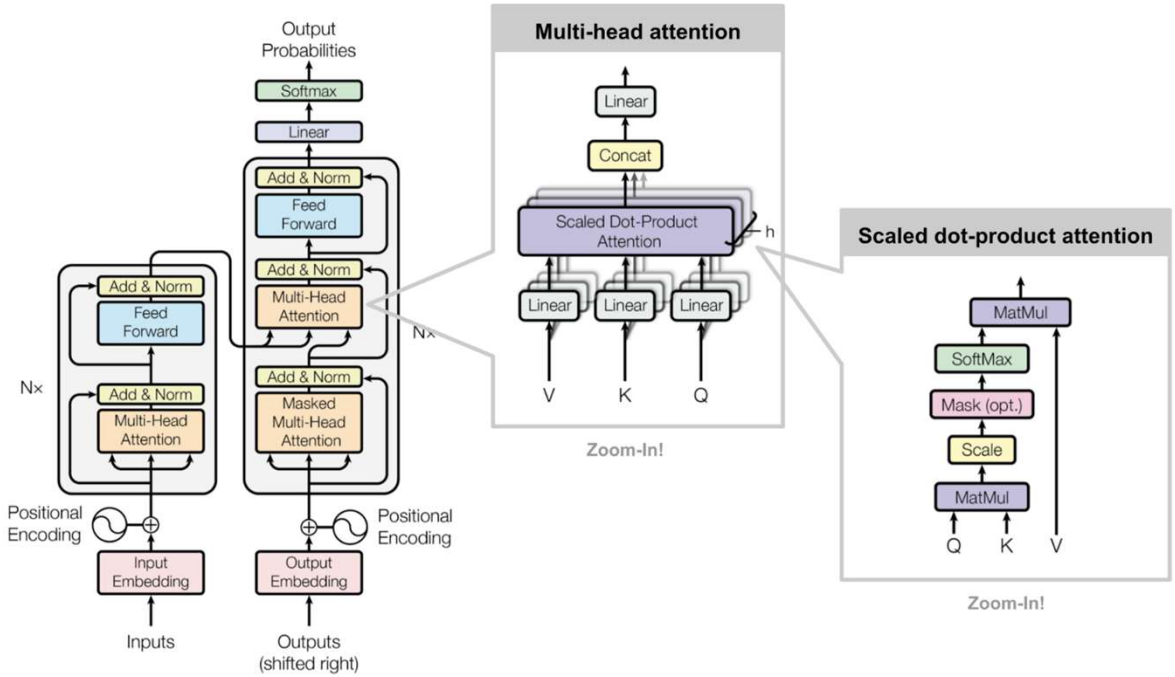
$$c_t = f_t \otimes c_{t-1} + i_t \otimes \tanh W \begin{pmatrix} x_t \\ h_{t-1} \end{pmatrix} \qquad f_t = \sigma\left(W_f \begin{pmatrix} x_t \\ h_{t-1} \end{pmatrix} + b_f \right)$$

Outline

- Machine Learning
- Neural networks
- Convolutional Neural Networks
- Recurrent Neural Networks
- **Transformers**

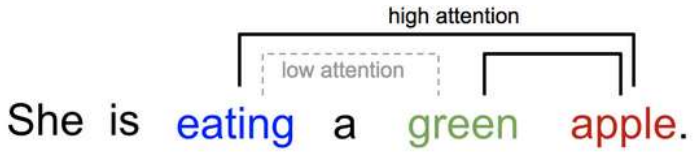
Transformers

- Natural language processing needs long-range relationships to be captured, but only those that matter
- Transformers provide a mechanism to focus attention on different relationships between data in a sequence
- Turns out to be useful in visual data too, where context can sometimes be important at the image level.



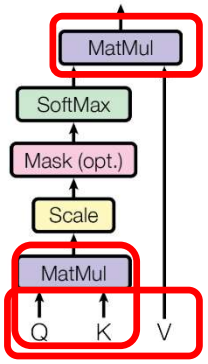
Transformer Basics

- Estimates how strongly correlated one element is to its neighbours in a sequence.
- 3 sets of inputs: Queries (Q) and Key (K), Value (V) pairs
- Attention map measures correlation between Q and K, and reweights V based on it
- Lots of uses in vision, point clouds, popping up everywhere



$$\text{Attention}(Q, K, V) = \text{softmax}(\frac{QK^T}{\sqrt{d_k}})V$$

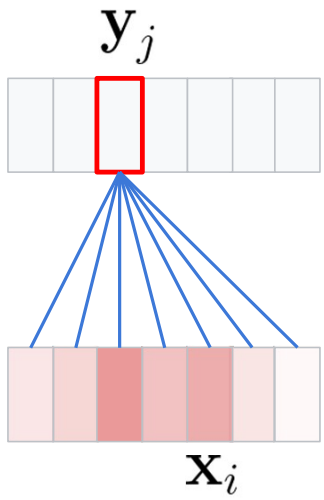
Scaled Dot-Product Attention



- [8] https://lilianweng.github.io/posts/2018-06-24-attention/
- [9] Vaswani, A., Shazeer, N., Parmar, N., Uszkoreit, J., Jones, L., Gomez, A. N., Kaiser, Ł., & Polosukhin, I. (2017). Attention is all you need. *Advances in Neural Information Processing Systems, 2017-Decem*(Nips), 5999–6009.

Self-attention

- Each of the tokens (=vectors) attends to all tokens
  - Extra tricks: learned key, query, and value projections, inverse-sqrt scaling in the softmax, and multi-headed attention (omit for simplicity)
- It's a set operation (permutation-invariant)
  - ...and hence need "position embeddings" to "remember" the spatial structure
- It's a global operation
  - Aggregates information from all tokens



$$\boldsymbol{\alpha}_j = softmax(\frac{K\mathbf{x}_1 \cdot Q\mathbf{x}_j}{\sqrt{d_K}}, \ldots, \frac{K\mathbf{x}_n \cdot Q\mathbf{x}_j}{\sqrt{d_K}})$$

$$\mathbf{y}_j = \sum_{i=1}^{n} \alpha_{ji} V\mathbf{x}_i$$

Simplified! Multi-headed attention not shown

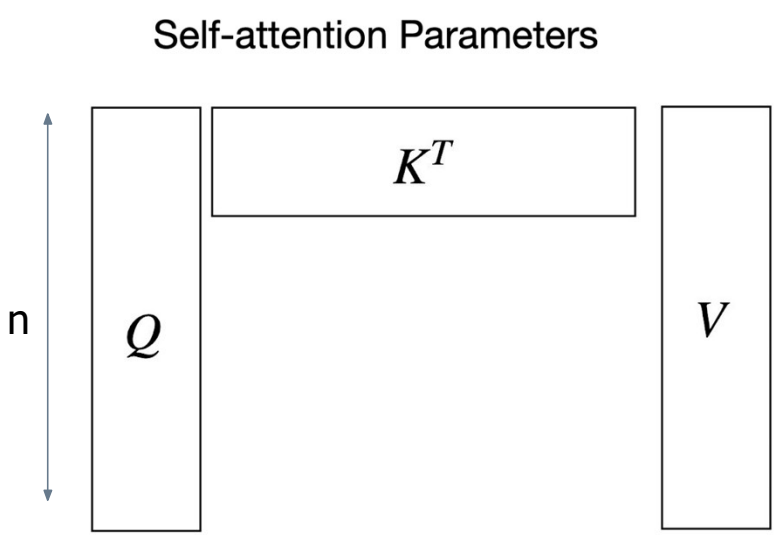
Transformer self-attention layer

Input: X (matrix of n embedding vectors, each dim m)

Parameters (learned): W_Q, W_K, W_V

Compute:

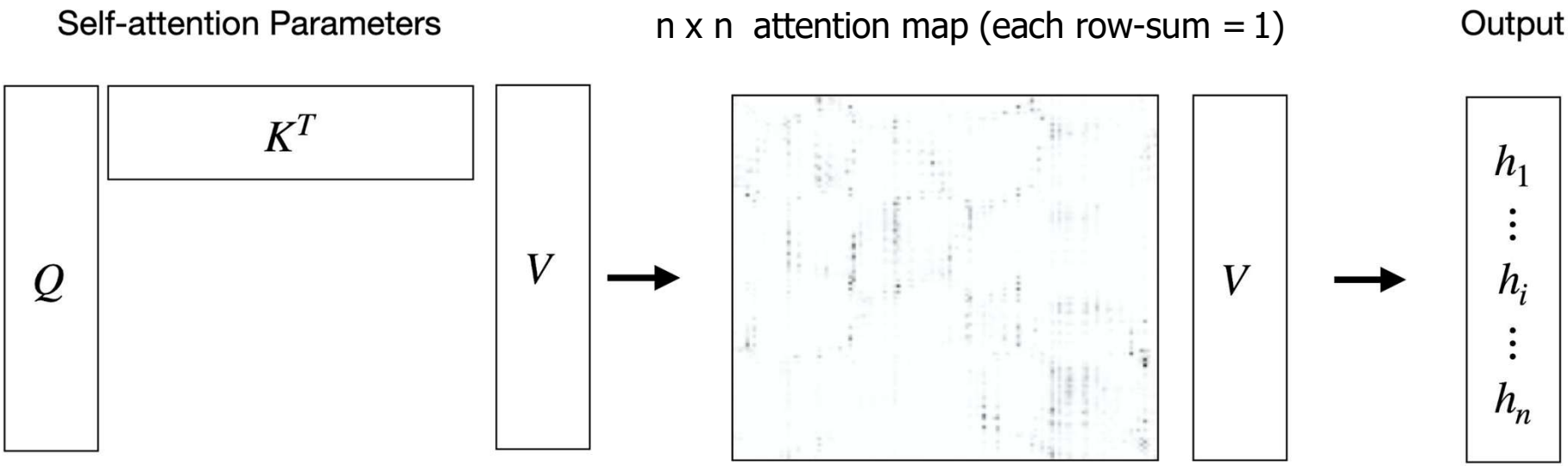
$$Q = XW_Q$$
$$K = XW_K$$
$$V = XW_V$$



$$Q, K, V \in \mathbb{R}^{n \times m}$$

$$QK^T \in \mathbb{R}^{n \times n}$$

Transformer self-attention

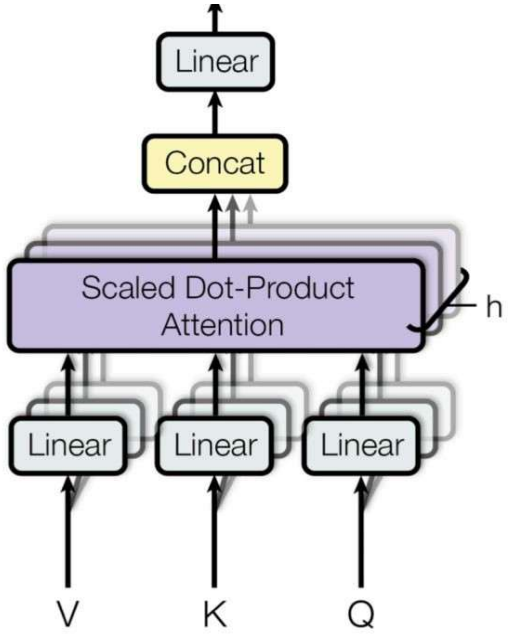


Output matrix H = $\mathrm{softmax}(\frac{1}{\sqrt{d}} QK^T) \cdot V$

Self-attention explicitly models interactions between all pairs of input embeddings

Multi-Head attention

- Learn *h* different linear projections of Q,K,V
- Compute attention separately on each of these *h* versions
- Concatenate and project the resultant vectors to a lower dimensionality.
- Each attention head can use low dimensionality



$$\text{MultiHead}(Q, K, V) = \text{Concat}(\text{head}_1, ..., \text{head}_\text{h})W^O$$

$$\text{where head}_\text{i} = \text{Attention}(QW_i^Q, KW_i^K, VW_i^V)$$

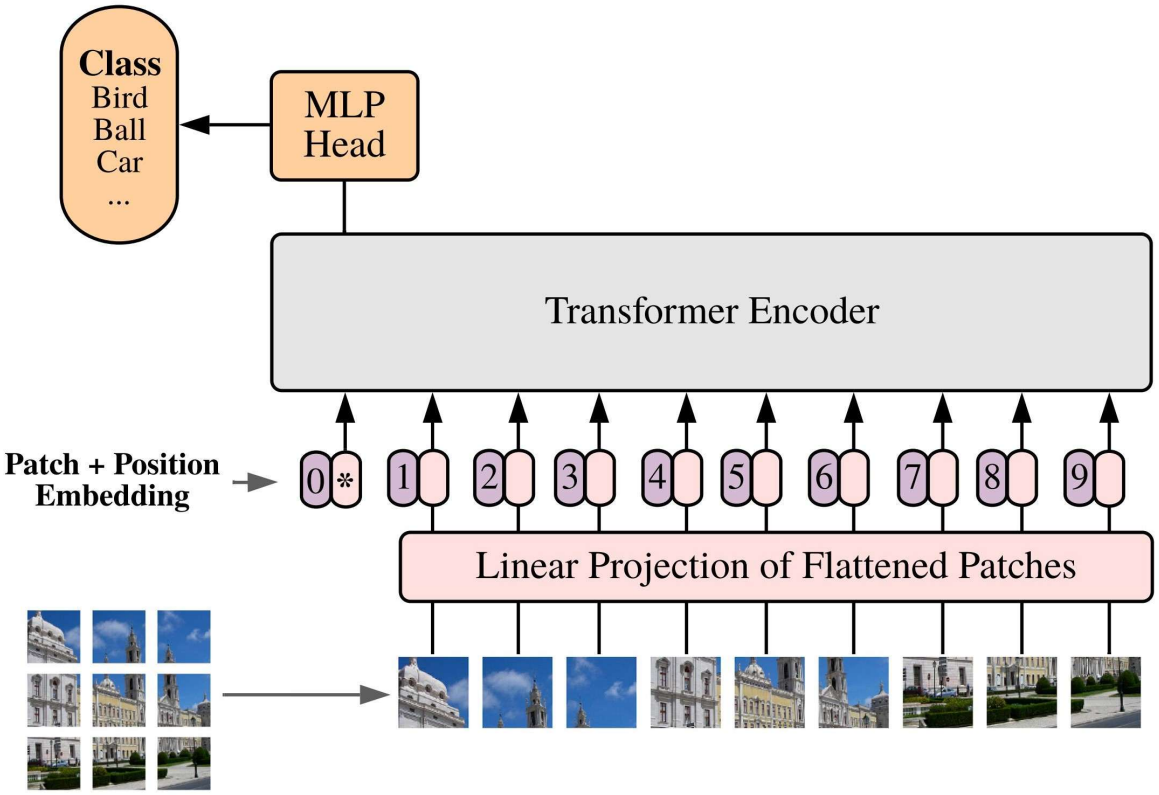
Transformers for vision?

- “LSTM → Transformer” ~ “ConvNet → ??? ”
- Issue with self-attention for vision: computation is quadratic in the input sequence length, quickly gets very expensive (with > few thousand tokens)
  - For ImageNet: 224x224 pixels → ~50,000 sequence length
  - Even worse for higher resolution and video

How can we deal with this quadratic complexity?

Vision Transformer (ViT)

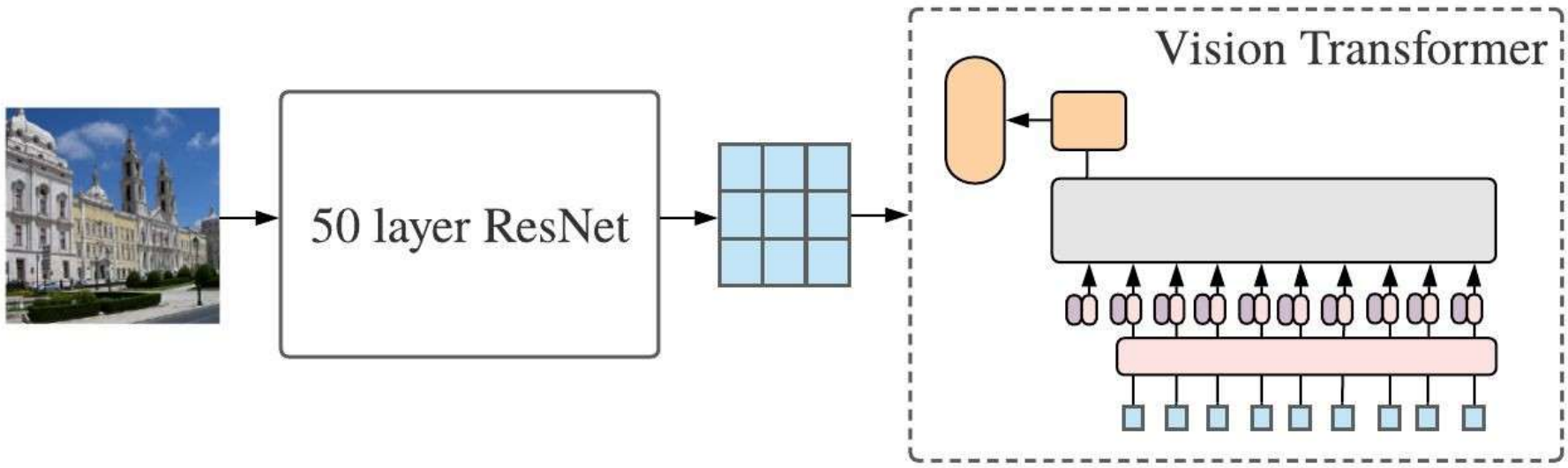
Idea: Take a transformer and apply it directly to image patches



Cordonnier et al., *On the Relationship between Self-Attention and Convolutional Layers*, ICLR 2020

Dosovitskiy et al., *An Image is Worth 16x16 Words: Transformers for Image Recognition at Scale*, ICLR 2021

ResNet-ViT Hybrid



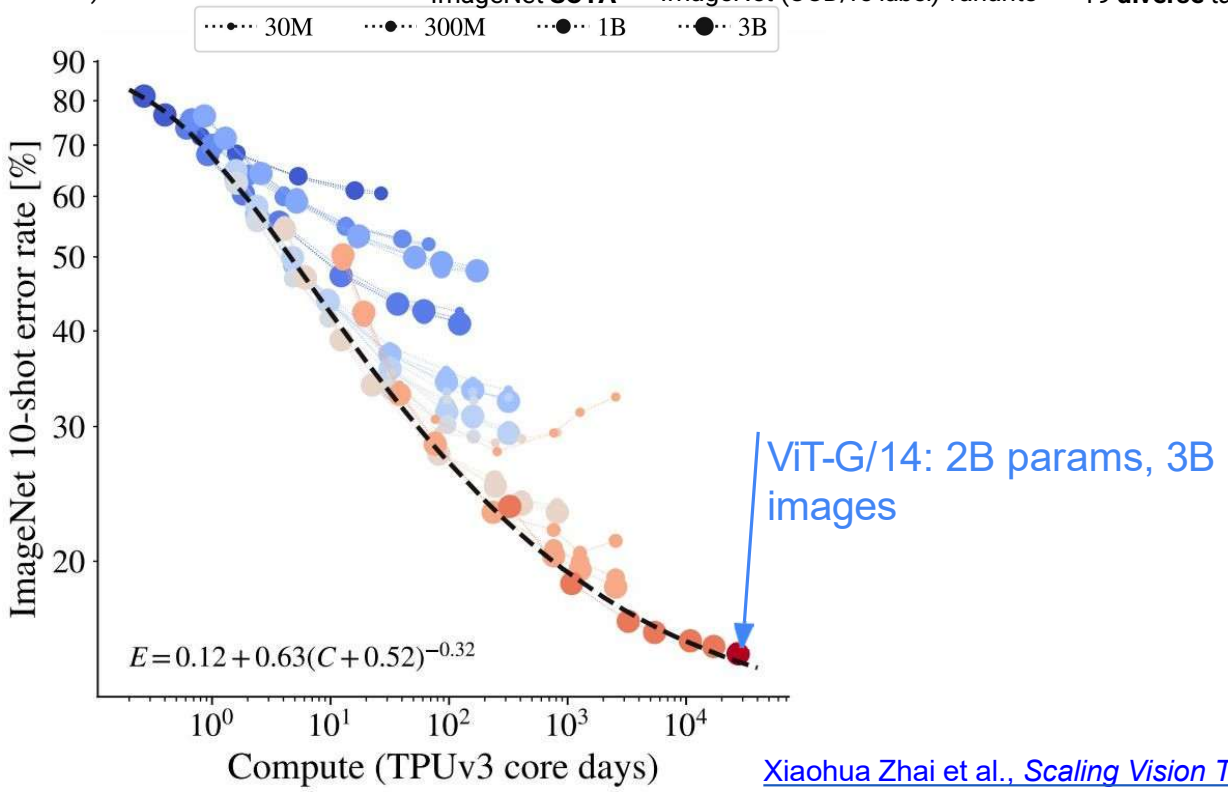
Bichen Wu et al. *Visual Transformers: Token-based Image Representation and Processing for Computer Vision*, arXiv 2020

Dosovitskiy et al., *An Image is Worth 16x16 Words: Transformers for Image Recognition at Scale*, ICLR 2021

Scaling Laws
How many images do you need for a big model & vice-versa?

84.86% top-1 accuracy on 10-shot ImageNet (**<1% of train set**)

ImageNet **SOTA** ImageNet (**OOD**/re-label) variants 19 **diverse** tasks



ViT-G/14: 2B params, 3B images

Xiaohua Zhai et al., *Scaling Vision Transformers*, arXiv 2021

Summary

- Machine learning is an important and powerful tool for extracting understanding from input and relies on converting patterns in input to features
- Learning can be thought of as a generalization of maximum likelihood estimation, where the task is not just to fit existing data but instead to build a model for future data from the same input source.
- Neural Networks are extremely effective at a wide range of learning tasks, and use back propagation to
- Convolutional Neural Networks are particularly well suited to processing sensor data, due to inherent spatial invariance and ability to extract low-dimensional representation from high-dimensional data
- Sequential data (time-series, NLP) need long-range relationships to be maintained, and so recurrent networks and transformers have been proposed to address this